

Image Segmentation

Ranga Rodrigo

Slides mainly from Yuri Boykov, Svetlana Lazebnik,
Alexei Efros, Steven Seitz and others

Contents

- Introduction
- Thresholding
- Region growing
- k-means
- Mean-shift
- Live wire
- Normalized cuts
- Introduction to segmentation with deep networks

What is Image Segmentation

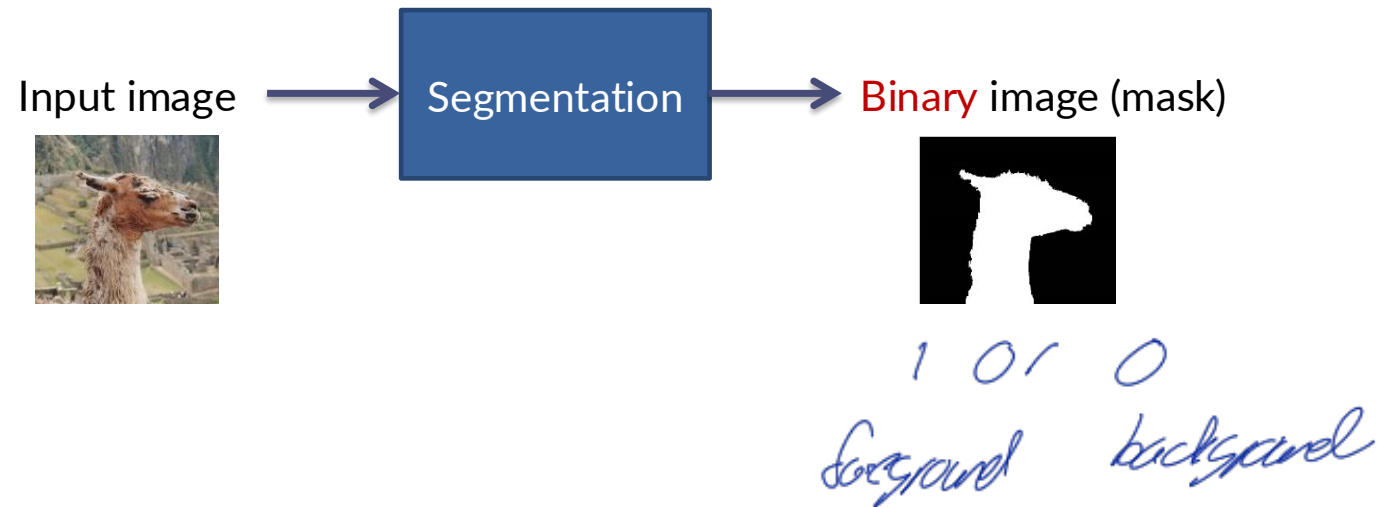
- Simplifying an image to meaningful pixel groupings.
- Partitioning an image into parts for deriving meaning or further analysis based on some **homogeneous visual characteristics**.
- Process of assigning a **label** to every pixel in an image such that pixels with the same label share certain visual characteristics.

Input and Output in Segmentation

Segmentation Output (Semantic)



Binary Segmentation Output





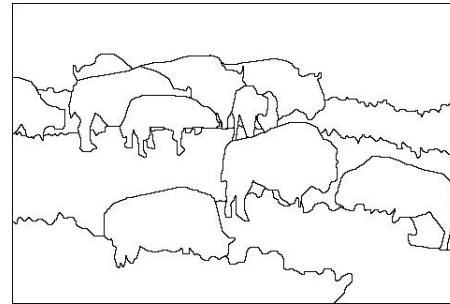
Svetlana Lazebnik

The Goals of Segmentation

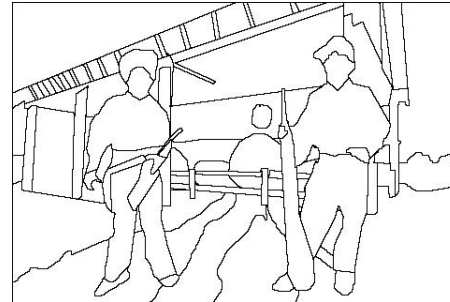
- Separate image into coherent “objects”
 - “Bottom-up” or “top-down” process?
 - Supervised or unsupervised?



image



human segmentation



Berkeley segmentation database:

<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench/>

Svetlana Lazebnik

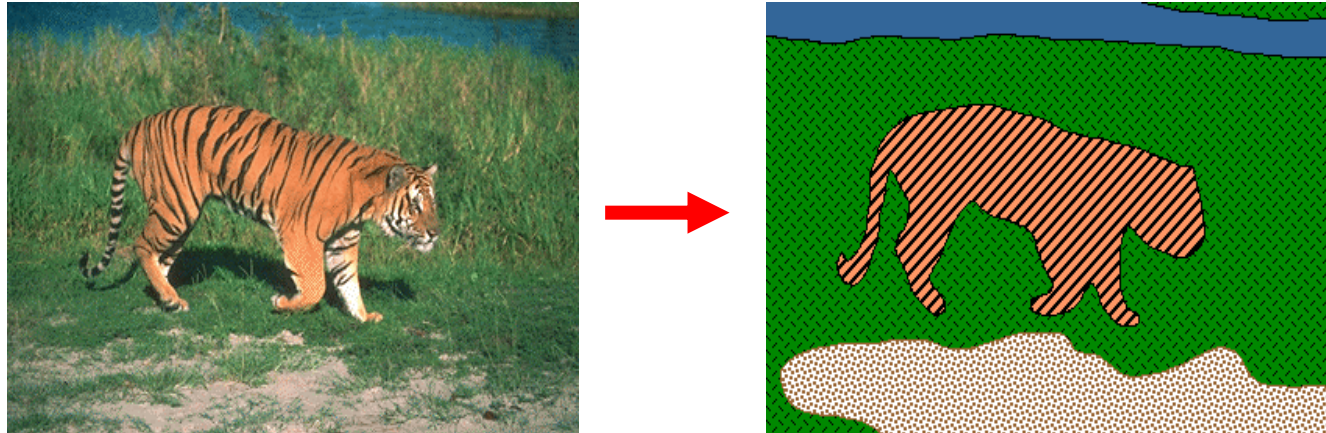
The Goals of Segmentation

Interactive segmentation for graphics



Svetlana Lazebnik

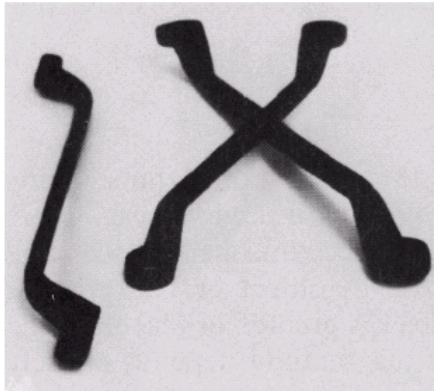
Goal: Extract “Blobs”



- What are “blobs”?
 - Regions of an image that are somehow coherent
- Why?
 - Object extraction, object removal, compositing, etc.
 - But are “blobs” objects?
 - No, not in general.

Blob Coherence

Simplest way to define blob coherence is as similarity in brightness or color:



The tools become blobs



The house, grass, and sky make different blobs

Approaches to Segmentation

- Segmentation as clustering
- Segmentation as graph partitioning
- Segmentation as labeling

Thresholding

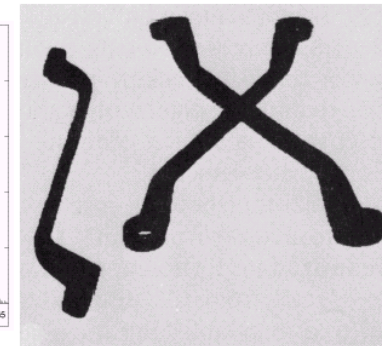
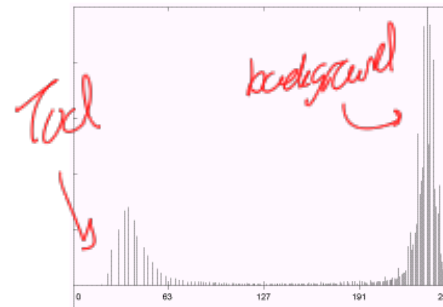
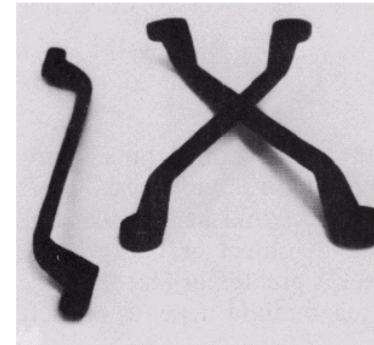
- Basic segmentation operation:

size of image

$$\text{mask}(x,y) = 1 \text{ if } \text{im}(x,y) > T$$

$$\text{mask}(x,y) = 0 \text{ if } \text{im}(x,y) < T$$

- T is threshold
 - User-defined
 - Or automatic
- Same as histogram partitioning:



easy to segment when there is a separation

(intra class)

How do we get T?

set T=0

FG
BG

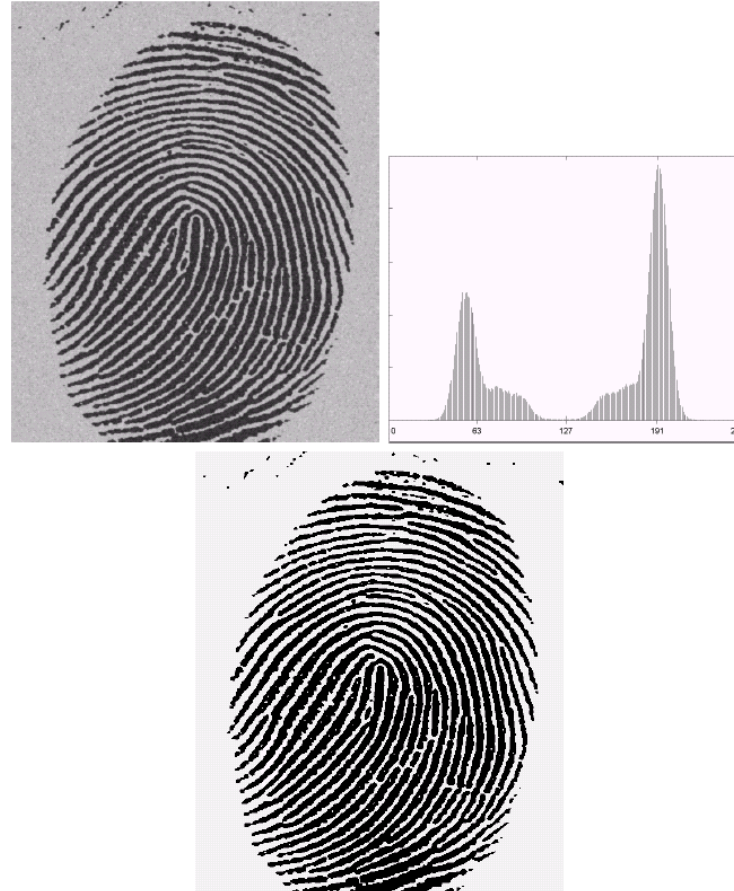
*} try to minimize var in a class
maximize var between class (inter class)*

Repeat for T=0 to 255

find T where the requirement is satisfied best.

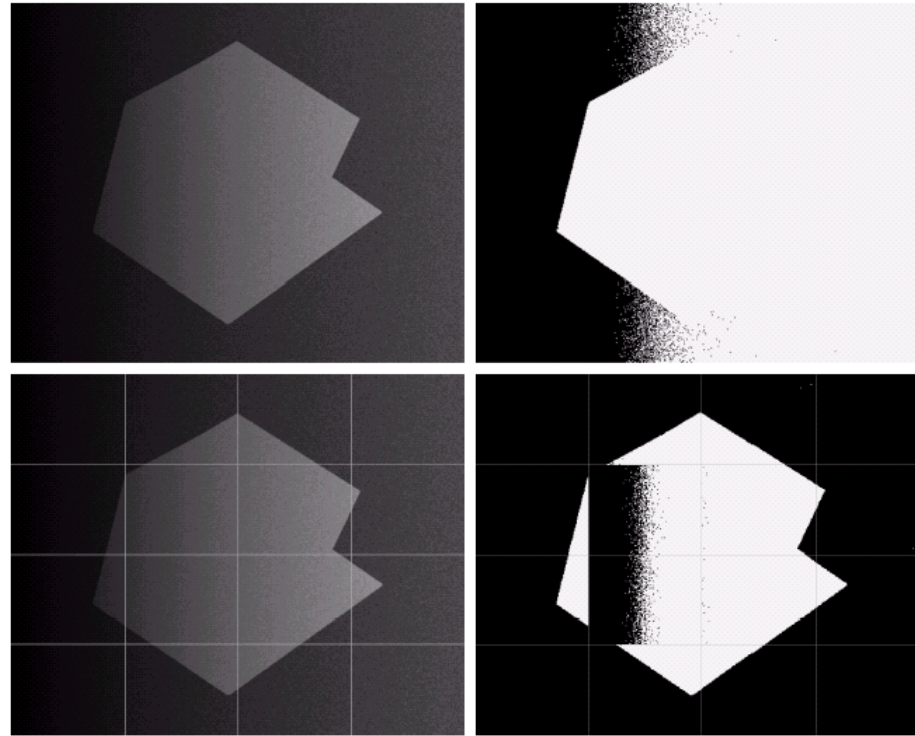
good for Bimodal histogram image

Thresholding: Successful Cases



What are potential Problems?

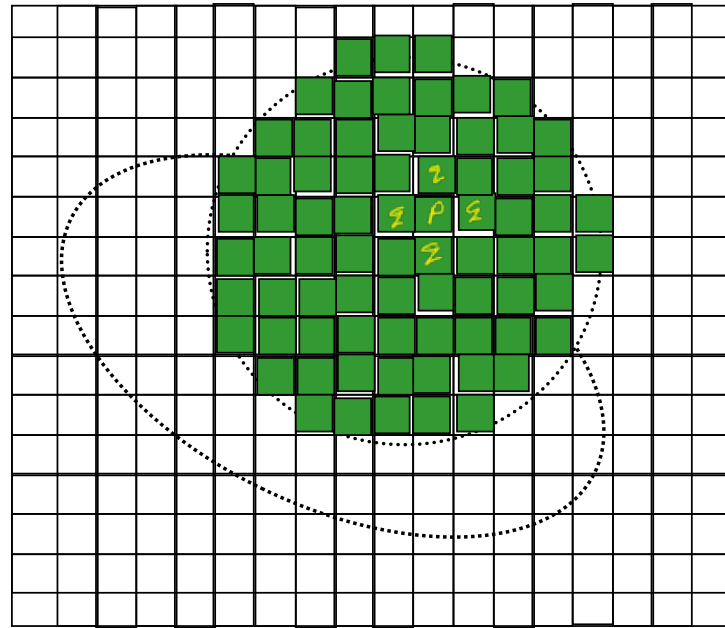
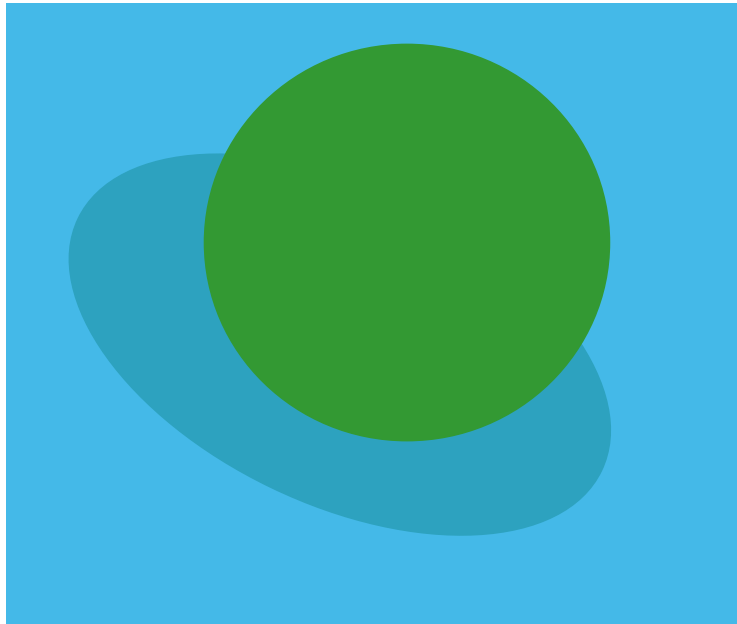
Thresholding: Unsuccessful Cases



Due to
shading we
cannot do
with const
TH

Adaptive thresholding

Region Growing



*This give
me
continuous
region*

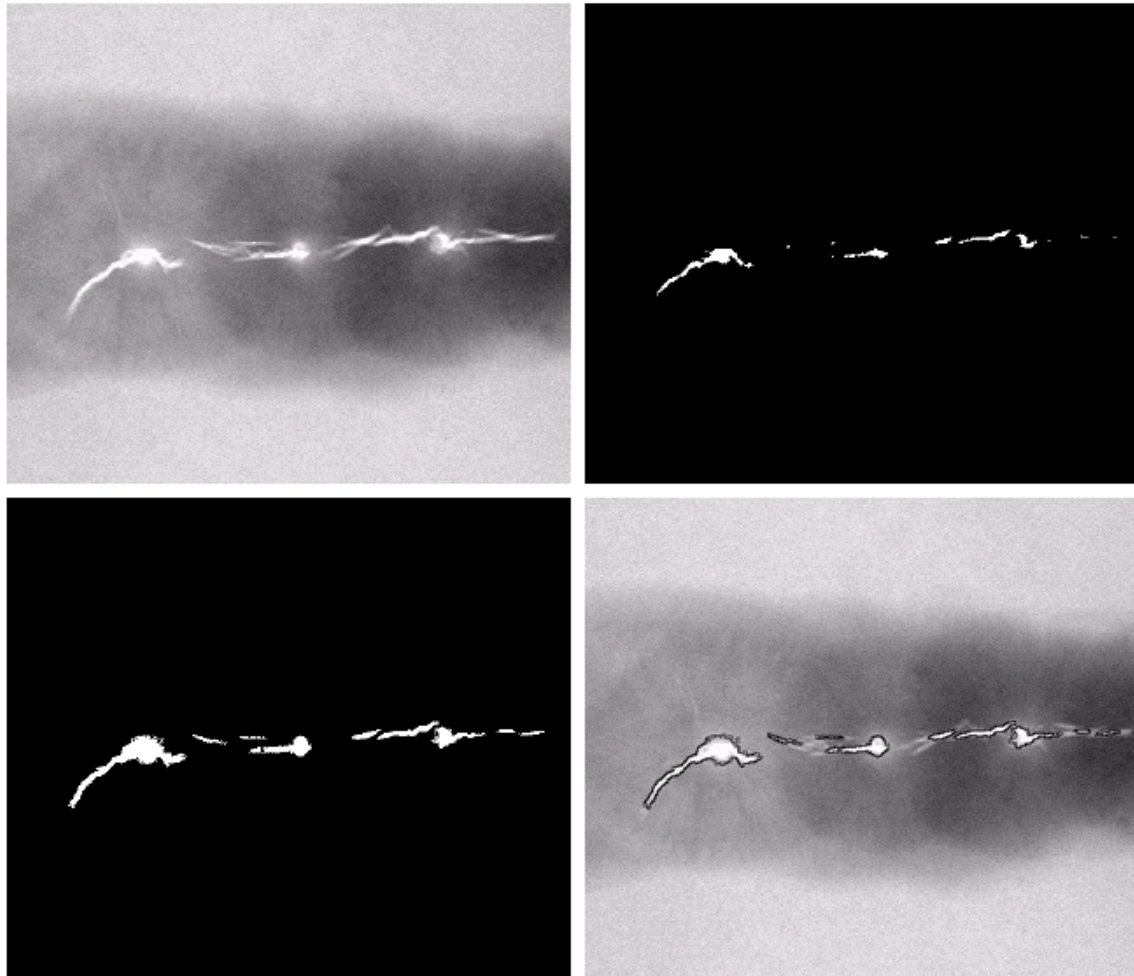
- Start with initial set of pixels K (initial seed(s))
- Add to pixels p in K their neighbors q if $|I_p - I_q| < T$
- Repeat until nothing changes
- Is this same as global threshold?

*can change to get
diff effects*

intensity

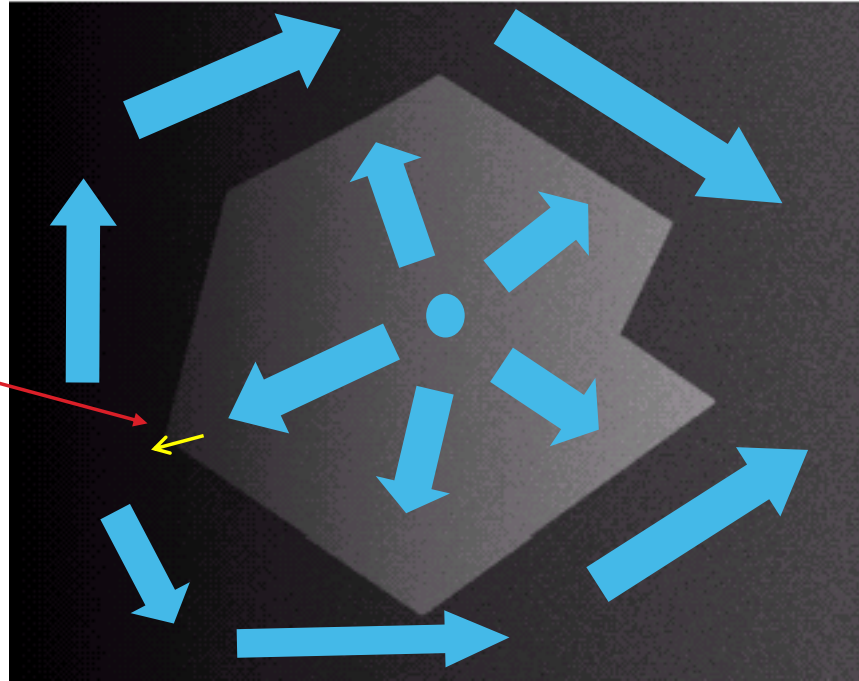
*start with a new circle
if intensity TH between pixel & < TH add it to our group
now & becomes P.
repeat
can implement using stack*

Region Growing Results



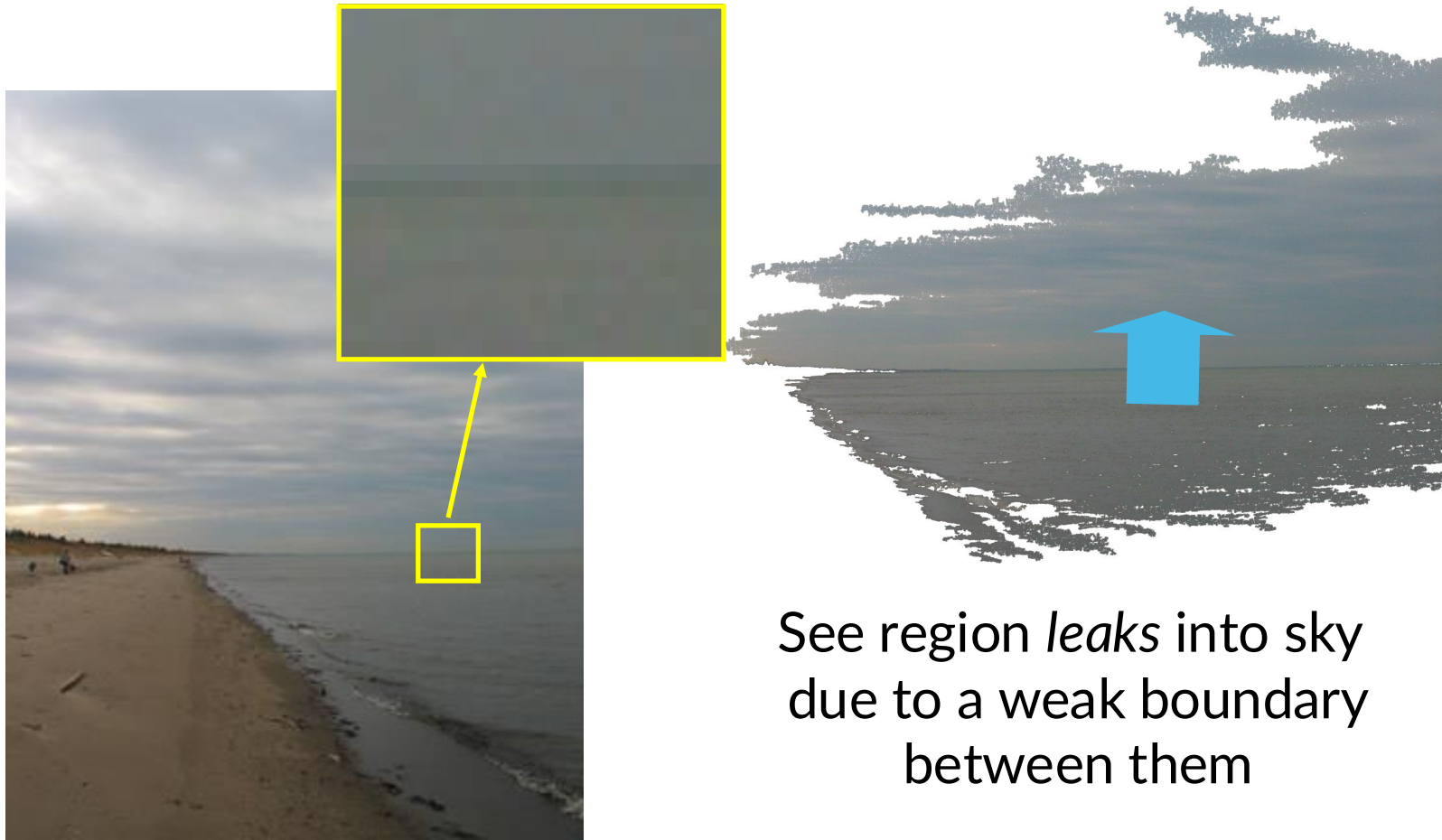
Problems with Region Growing

Region growth may “leak”
through a single **weak spot**
in the boundary



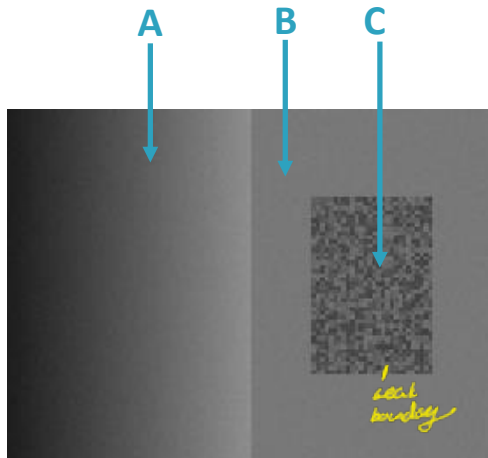
If there is at least
one weak point at
boundary, it will leak

Region Growing



Example

This image has three perceptually distinct regions



Difference along border between A and B is less than differences within C

Q: Where would **image thresholding** fail?

A: Region A would be divided in two sets and region C will be split into a large number of arbitrary small subsets

Q: Where would **region growing** fail?

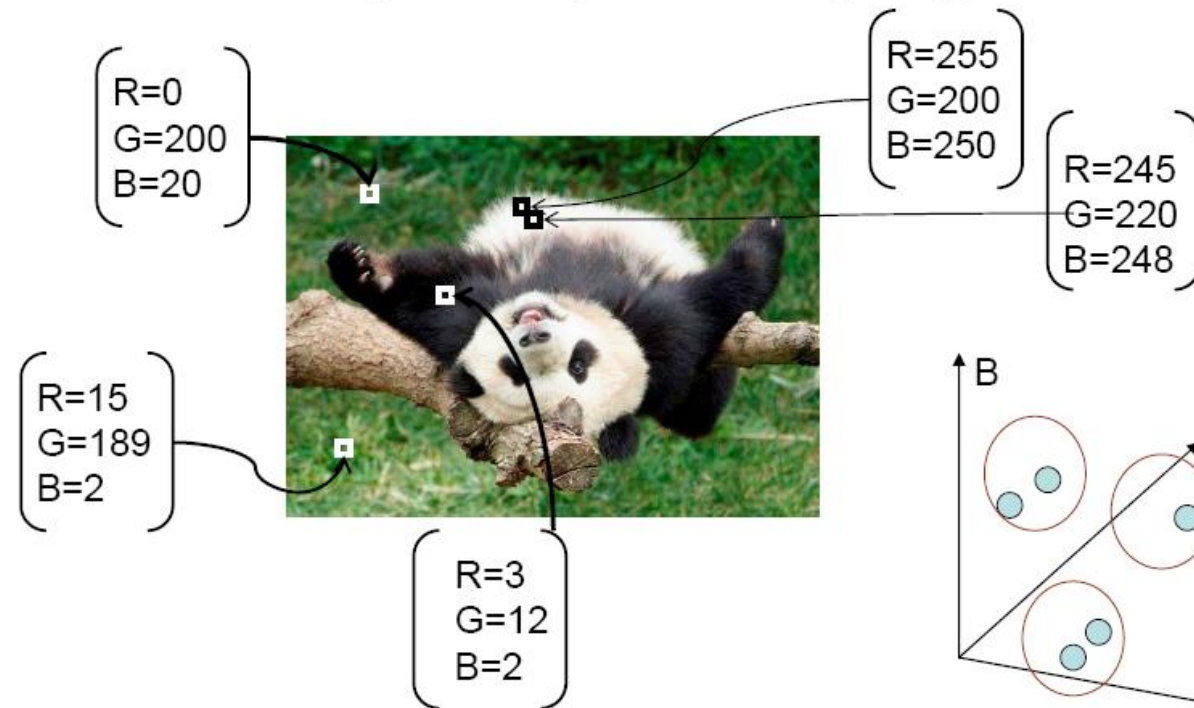
A: Either A and B are merged or region C is split into many small subsets
Also, B and C are merged

Region Growing

- Pros:
 - Simple
 - Invariant of the shape of the structure to be segmented
 - Ability to specify seeds, similarity criteria
- Cons:
 - Leaking at weak boundary points
 - Seed(s), minimum region threshold, similarity threshold to be specified

Segmentation as Clustering

- Cluster similar pixels (features) together



issue $\Rightarrow$ black in several regions will cluster together.

Source: K. Grauman

Segmentation as Clustering

- K-means clustering based on intensity or color is essentially vector quantization of the image attributes
 - Clusters don't have to be spatially coherent

Image



Intensity-based clusters



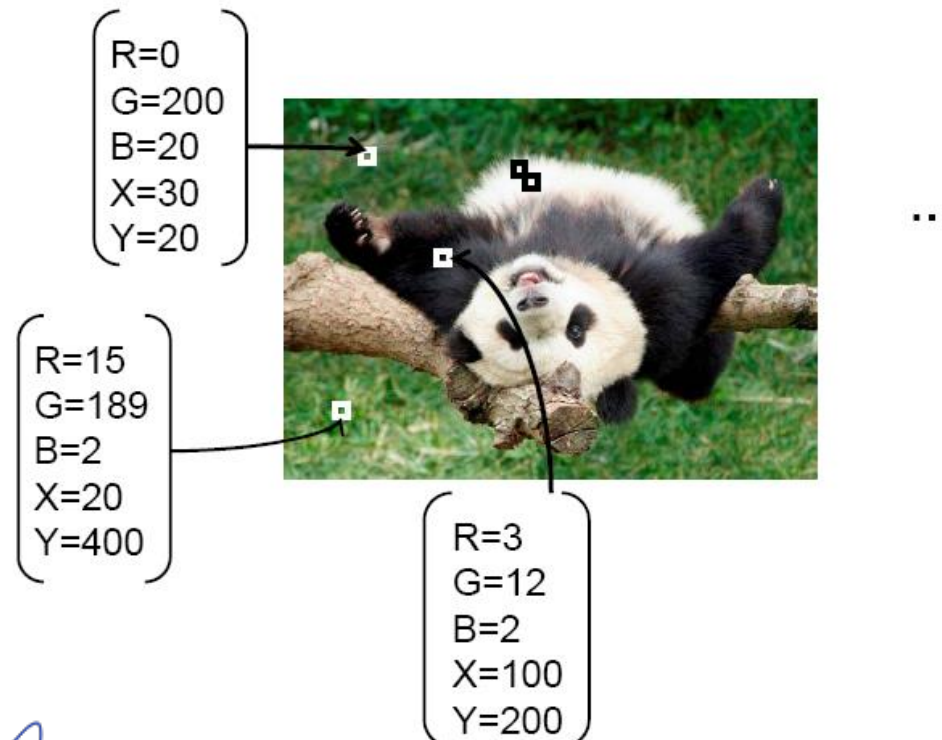
Color-based clusters



Slide by Svetlana Lazebnik

Segmentation as Clustering

- Cluster similar pixels (features) together



To fix issue we introduce X and Y
So based on both colour and distance.

Source: K. Grauman

Segmentation as Clustering

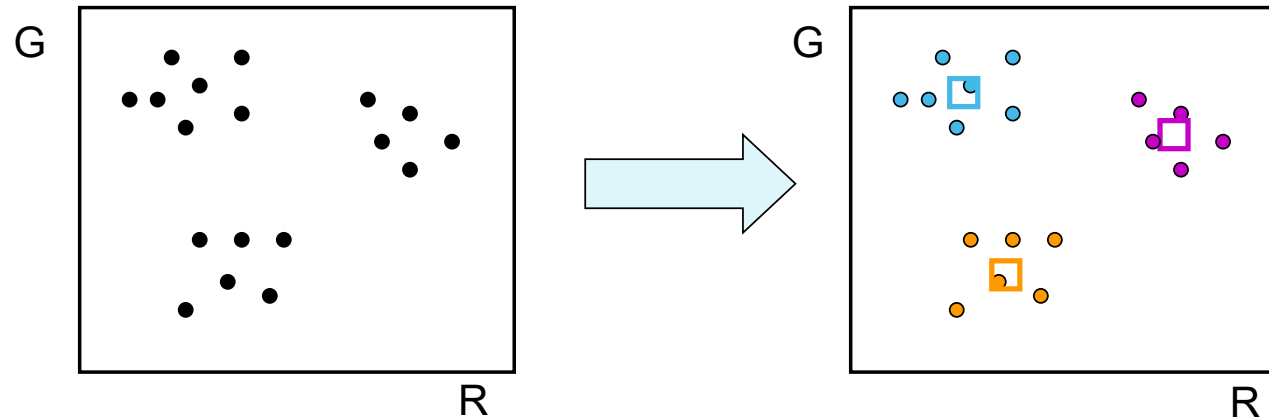
- Clustering based on (r, g, b, x, y) values enforces more spatial coherence



Slide by Svetlana Lazebnik

Clustering

- How to choose the representative colors?
 - This is a clustering problem!



Objective

- Each point should be as close as possible to a cluster center
 - Minimize sum squared distance of each point to closest center

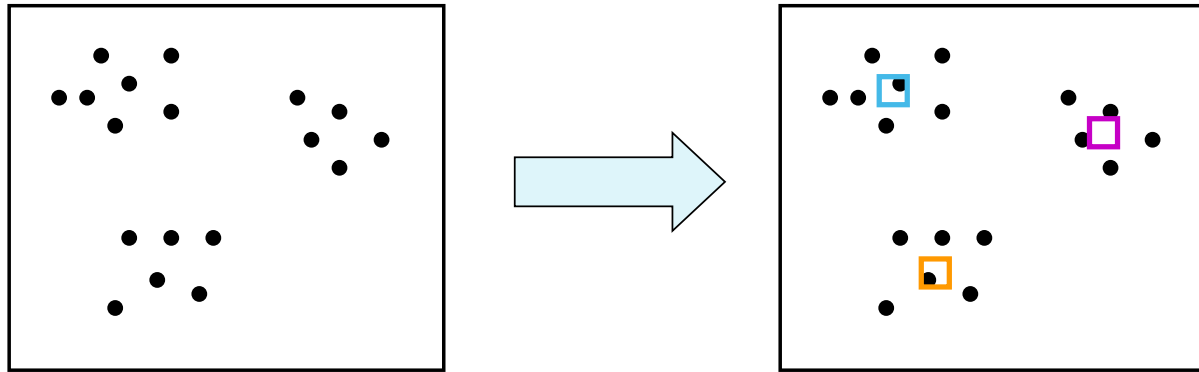
$$\sum_{\text{clusters } i} \sum_{\text{points } p \text{ in cluster } i} \|p - c_i\|^2$$

Slide by Steve Seitz

*Run until the points stop changing
cluster centers*

Break It Down into Subproblems

- Suppose I tell you the cluster centers c_i
 - Q: how to determine which points to associate with each c_i ?
 - A: for each point p , choose closest c_i



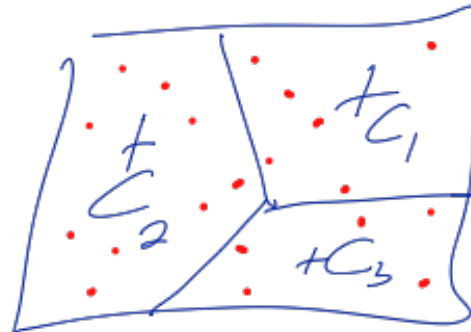
Suppose I tell you the points in each cluster

- Q: how to determine the cluster centers?
- A: choose c_i to be the mean of all points in the cluster

K-Means Clustering: Algorithm

1. Randomly initialize the cluster centers, $c_1, \dots, c_K$
2. Given cluster centers, determine points in each cluster
 - For each point p , find the closest c_i . Put p into cluster i
3. Given points in each cluster, solve for c_i
 - Set c_i to be the mean of points in cluster i
4. If c_i have changed, repeat Step 2

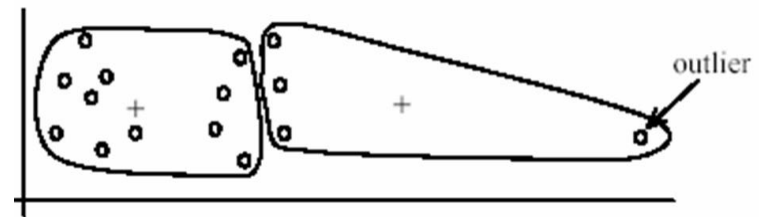
Suppose 10000 points
3 centers
To we have to
find dist 30000? No
We can find cluster
boundaries and use them



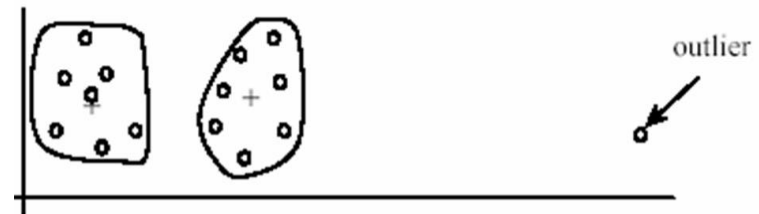
Text by Steve Seitz

K-Means for Segmentation

- Pros
 - Very simple method
 - Converges to a local minimum of the error function
- Cons
 - Memory-intensive
 - Need to pick K
 - Sensitive to initialization
 - Sensitive to outliers
 - Only finds “spherical” clusters



(A): Undesirable clusters



(B): Ideal clusters

Color-Based Blob Segmentation

- Automatic Histogram Partitioning

use case $\Rightarrow$ cartoonify

- Given image with N colors, choose K
- Each of the K colors defines a region
 - not necessarily contiguous
- Performed by computing color histogram, looking for modes
- This is what happens when you downsample image color range, for instance in Photoshop



Comaniciu and Meer 2002

*walls $\Rightarrow$ 1 color
grass $\Rightarrow$ 1 color etc*

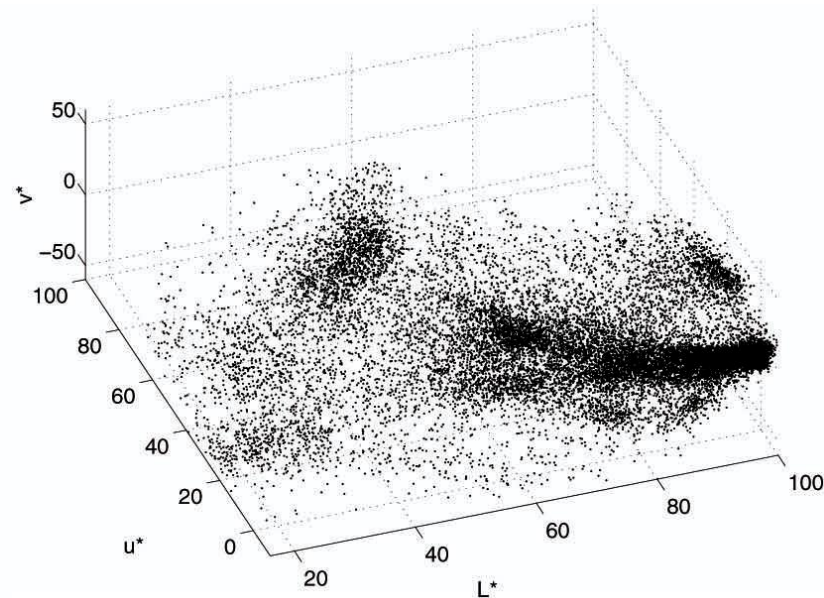
Mean-Shift Algorithm

- The mean shift algorithm seeks *modes* or local maxima of density in the feature space

image



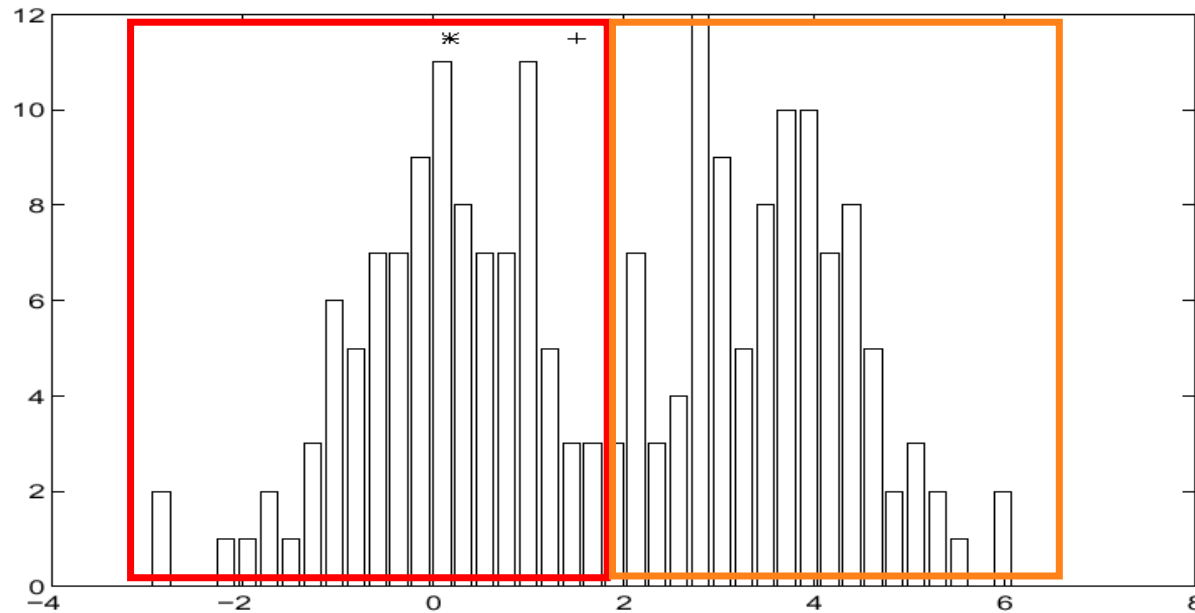
Feature space
($L^*u^*v^*$ color values)



LUV density map

Finding Modes in a Histogram

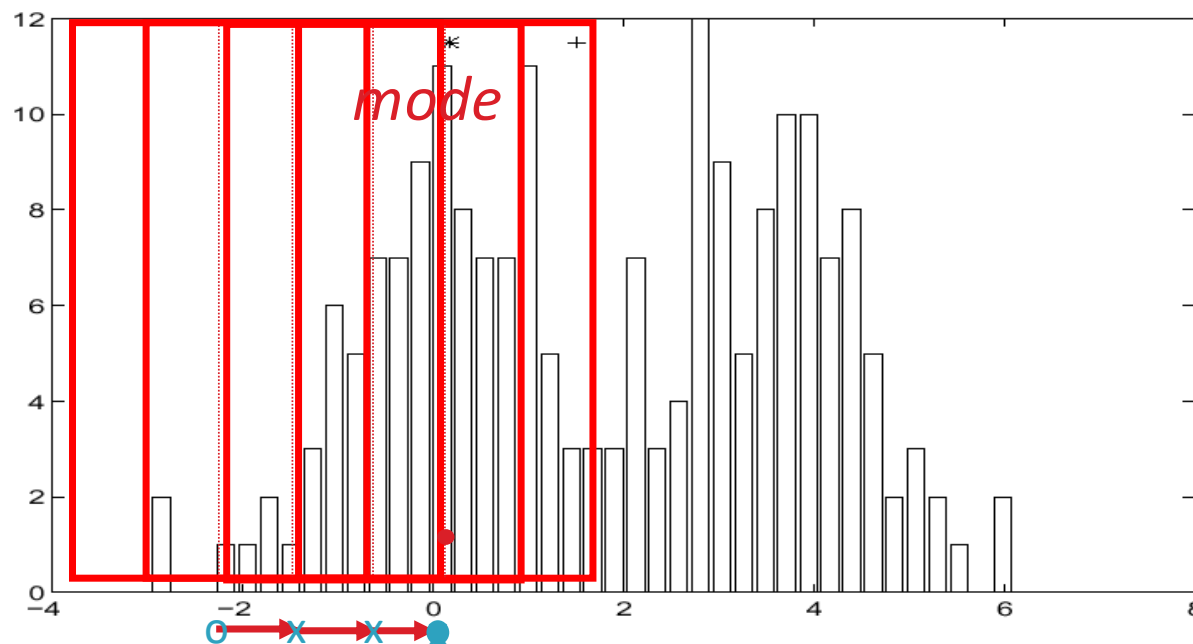
for each get 1D density map



- How Many Modes Are There?
 - Easy to see, hard to compute

2

Mean Shift [Comaniciu & Meer]



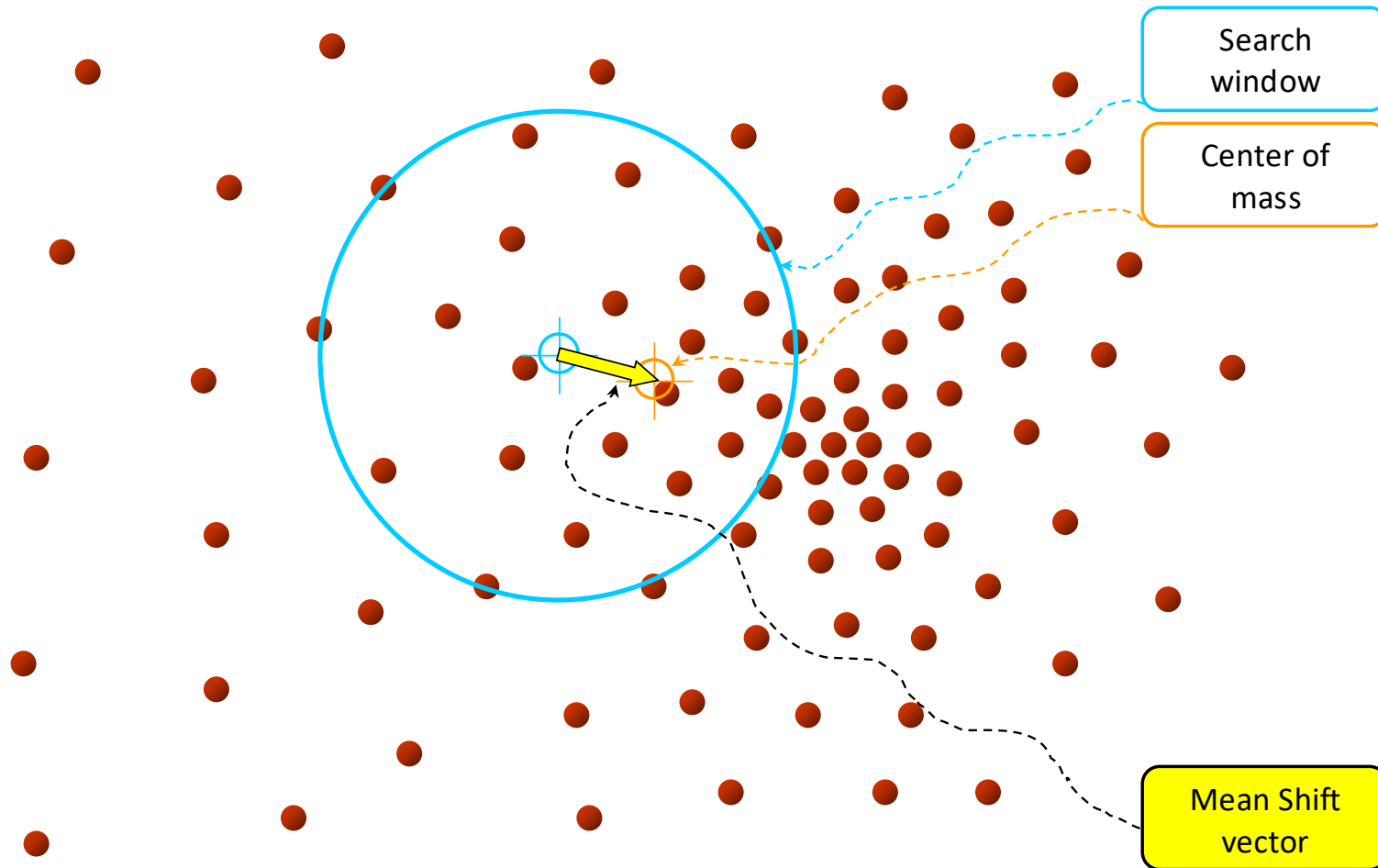
Iterative Mode
Search

1. Initialize random seed, and fixed window
2. Calculate center of gravity 'x' of the window (the "mean")
3. Translate the search window to the mean
4. Repeat Step 2 until convergence

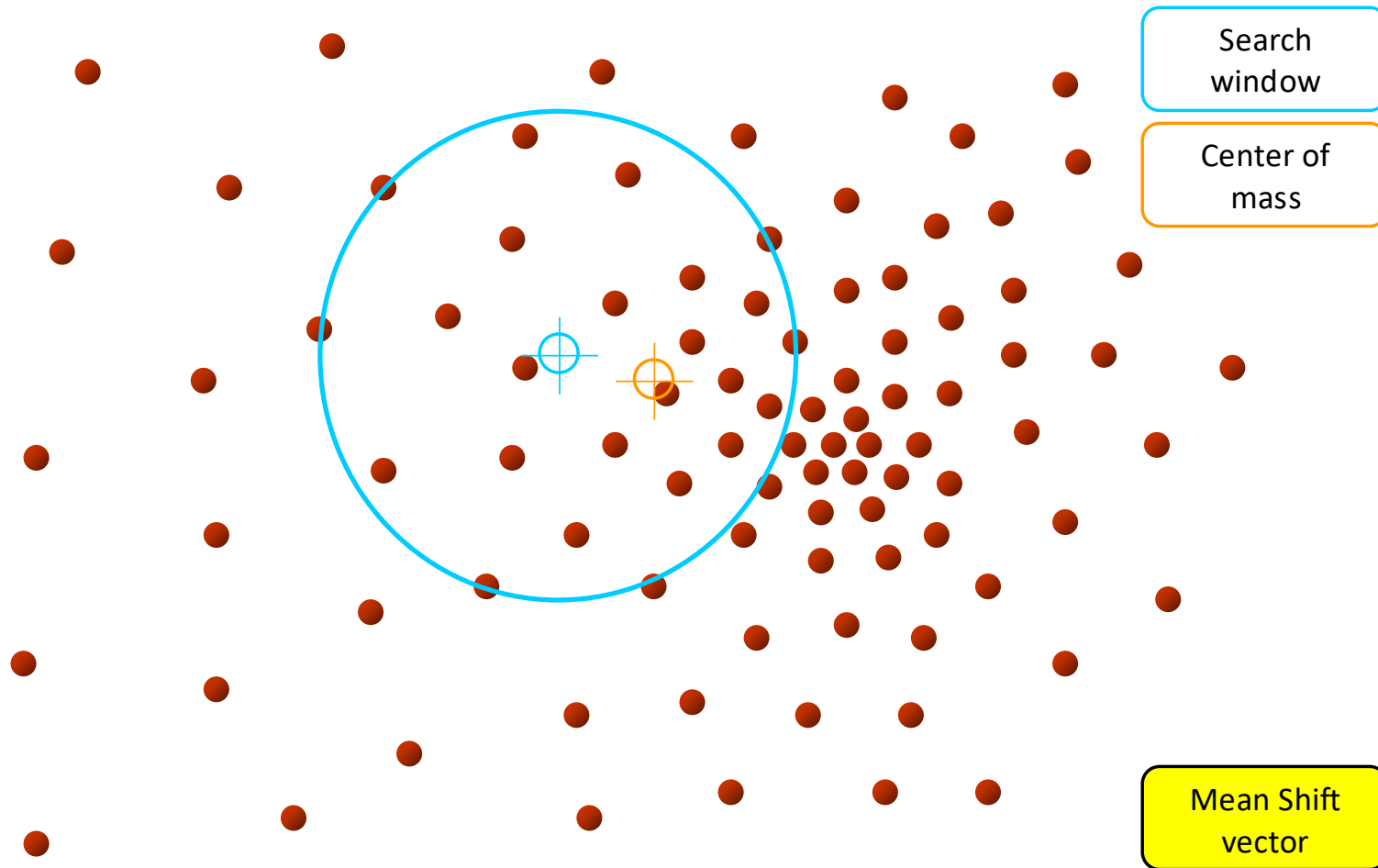
get a window with mean 0
find center of mass x
move 0 to x
repeat until $0 = x$

issues $\Rightarrow$ window size.
 $\Rightarrow$ is there any
real mode?

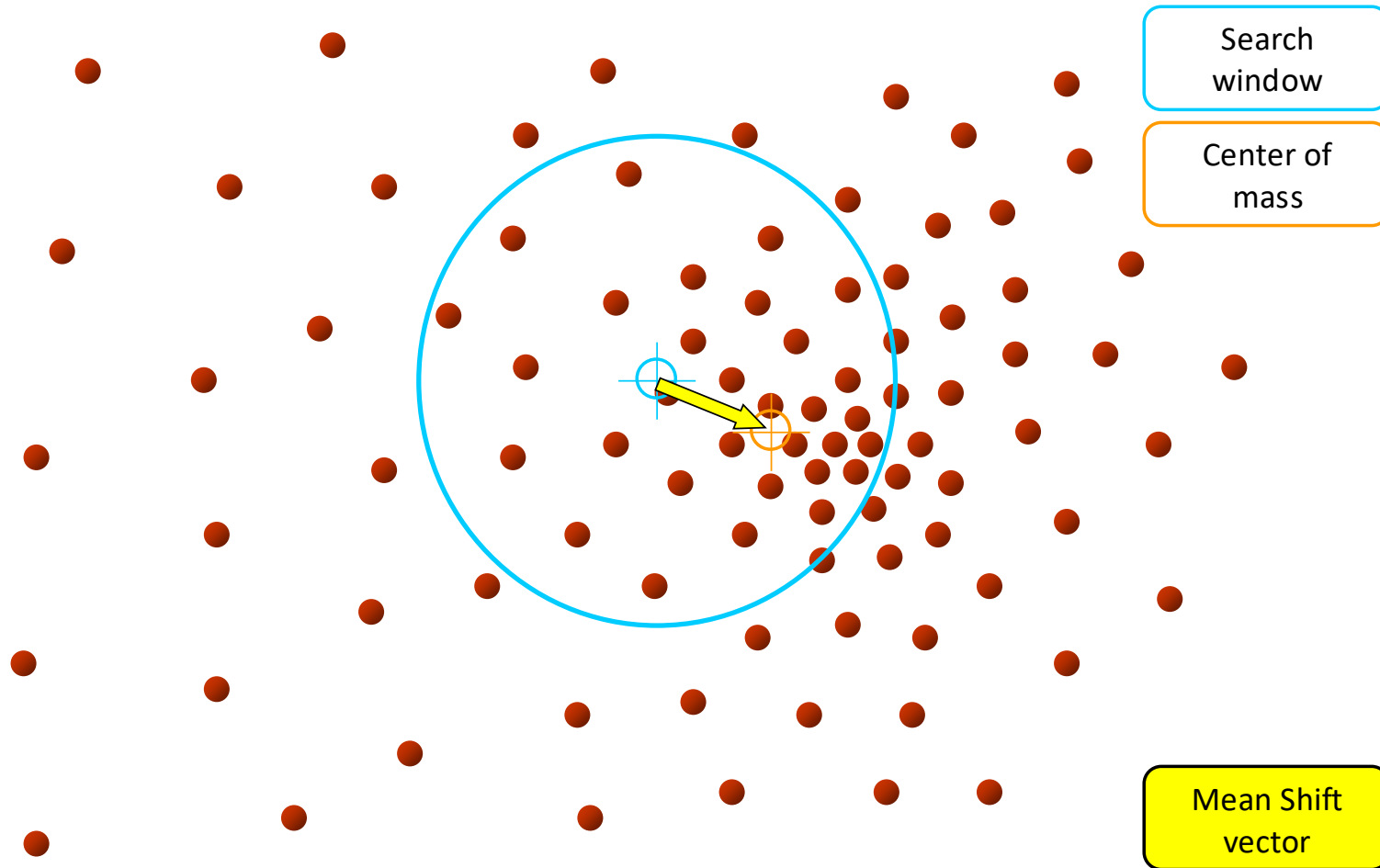
Mean Shift



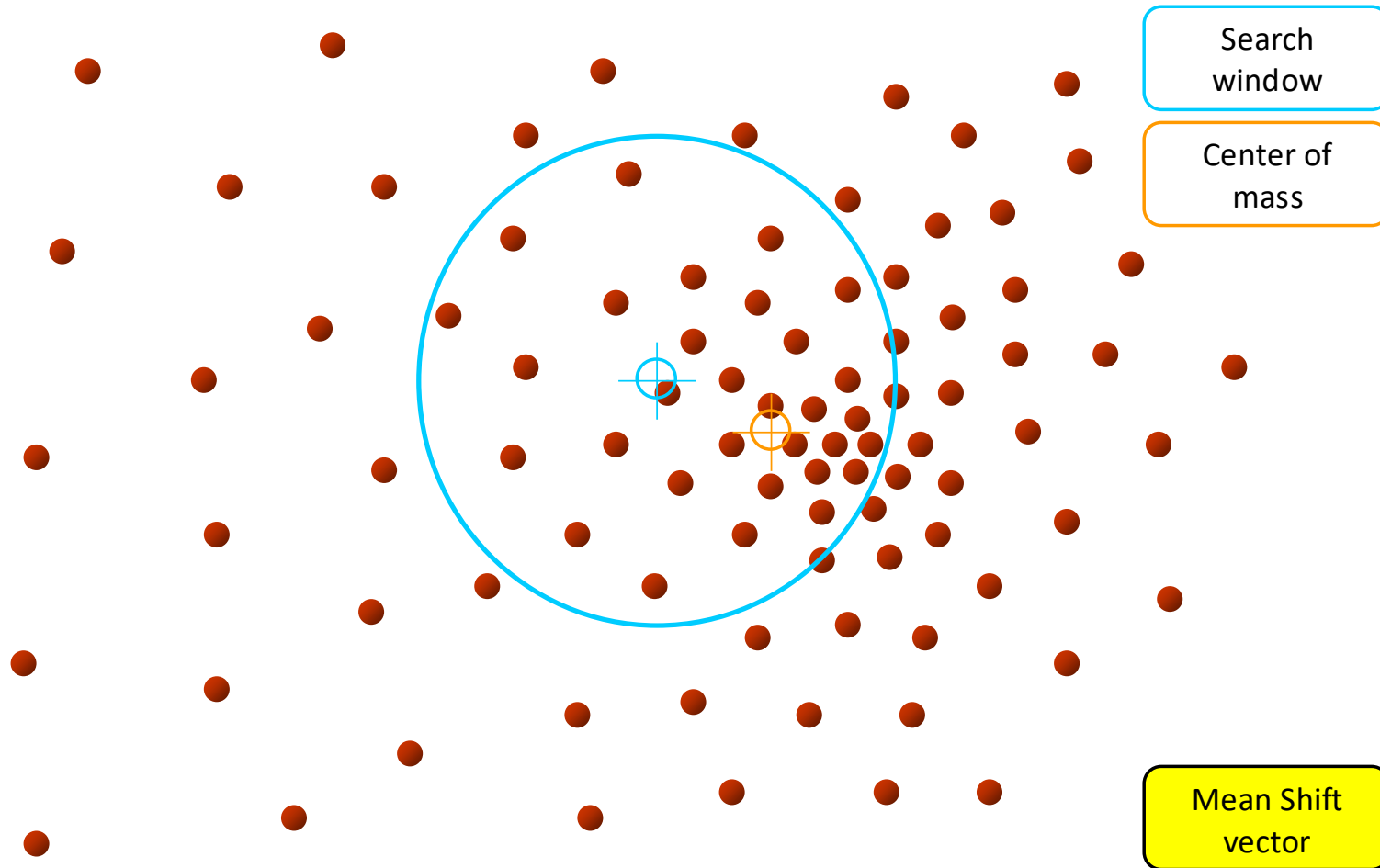
Mean Shift



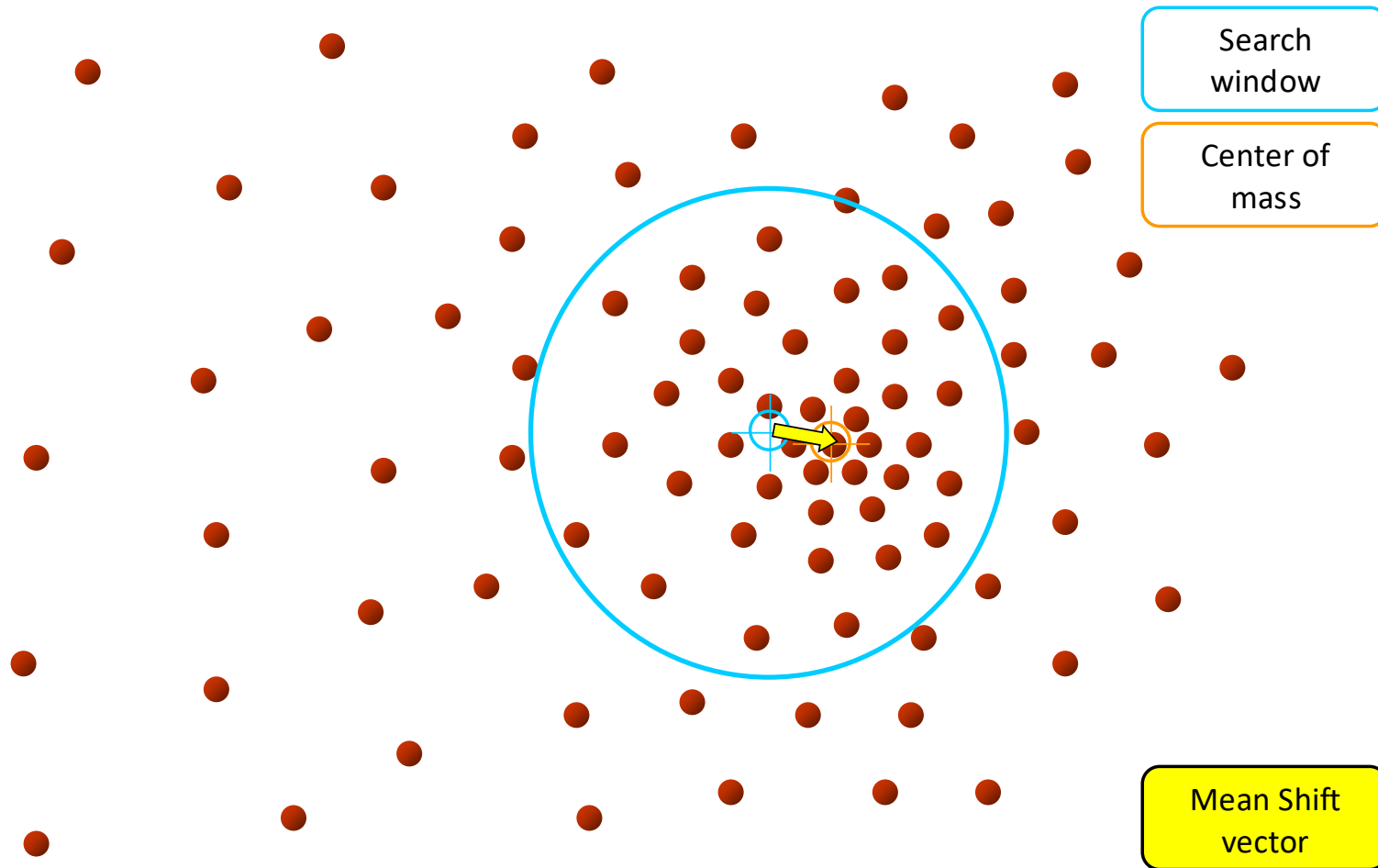
Mean Shift



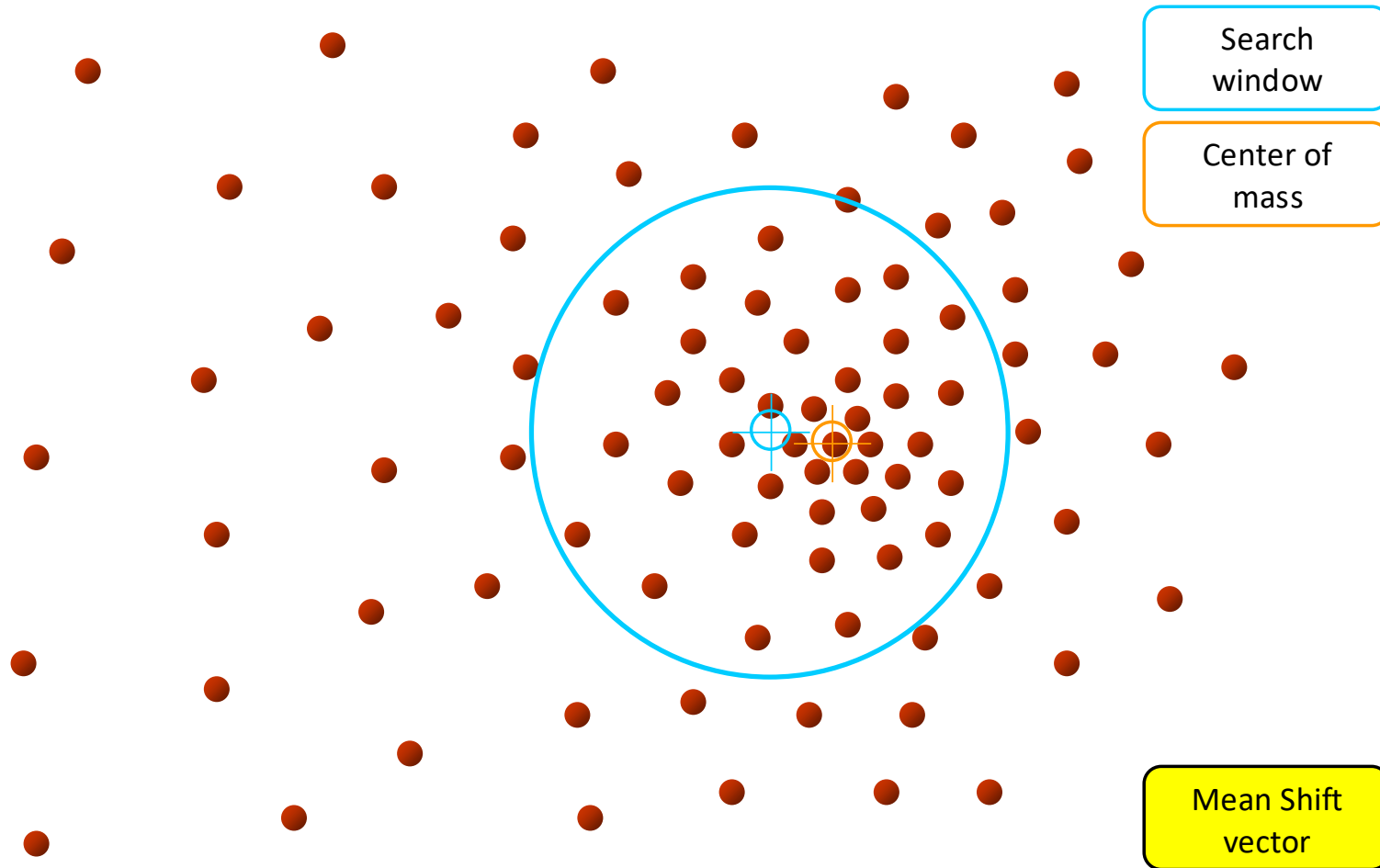
Mean Shift



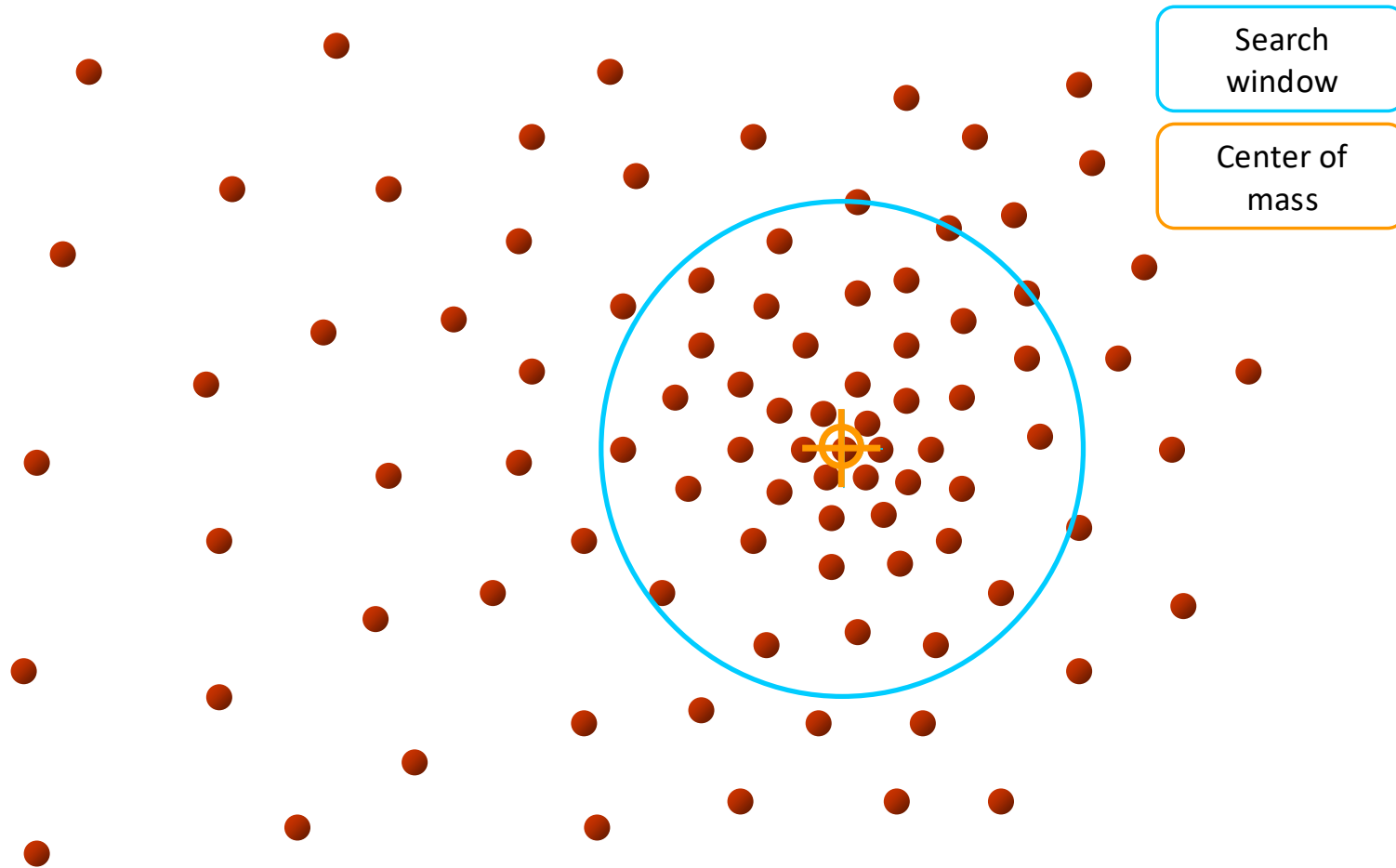
Mean Shift



Mean Shift



Mean Shift



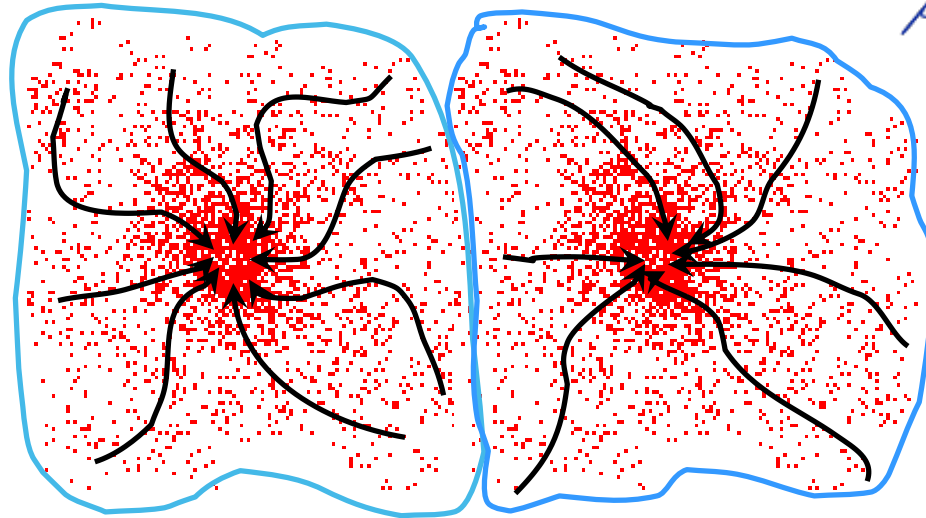
Slide by Y. Ukrainitz & B. Sarel

Can find location of maximum by this.
how to do segmentation now?

Mean-Shift Clustering

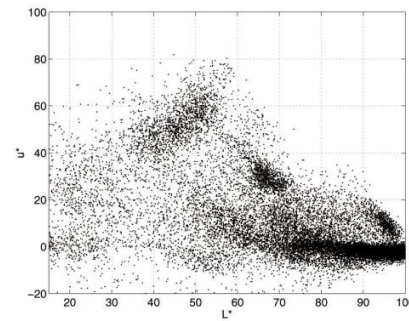
- Cluster: all data points in the attraction basin of a mode
- Attraction basin: the region for which all trajectories lead to the same mode

① Do mean shift from each pixel. The pixels that go the same peak belong to the same cluster.

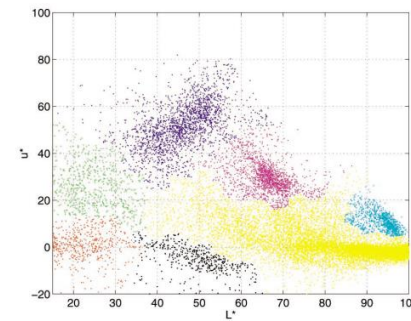


Mean-Shift: Cluster, Segment

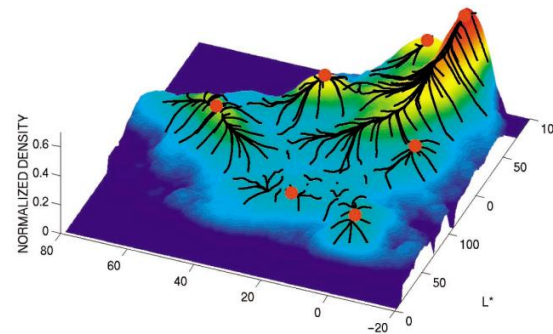
- Find features (color, gradients, texture, etc)
- Initialize windows at individual feature points
- Perform mean shift for each window until convergence
- Merge windows that end up near the same “peak” or mode



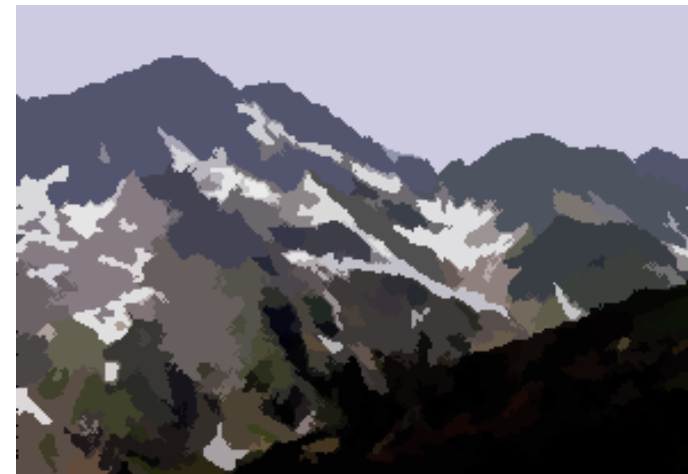
(a)



(b)



Mean-Shift Segmentation Results



<http://www.caip.rutgers.edu/~comanici/MSPAMI/msPamiResults.html>

Tucker Hermans

Mean-Shift Segmentation Results



Tucker Hermans

Mean Shift

- Pros:

- Does not assume shape on clusters
- One parameter choice (window size)
- Generic technique
- Find multiple modes

- Cons:

- Selection of window size
- Does not scale well with dimension of feature space

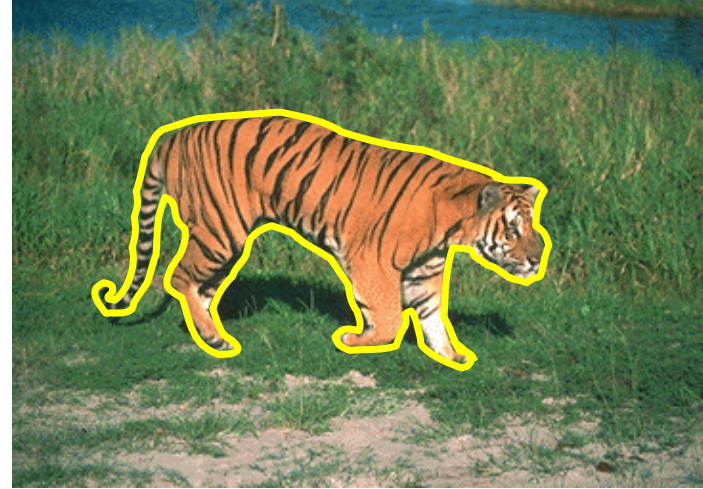
high dimensional histogram

Issues

- Although often useful, all these approaches work only some of the time, and are considered rather “hacky”.
- Can't even handle our tiger:
- Problem is that blobs != objects!



Extracting Objects



- How could this be done?

*if we had black stripes
different detector.*

Intelligent Scissors

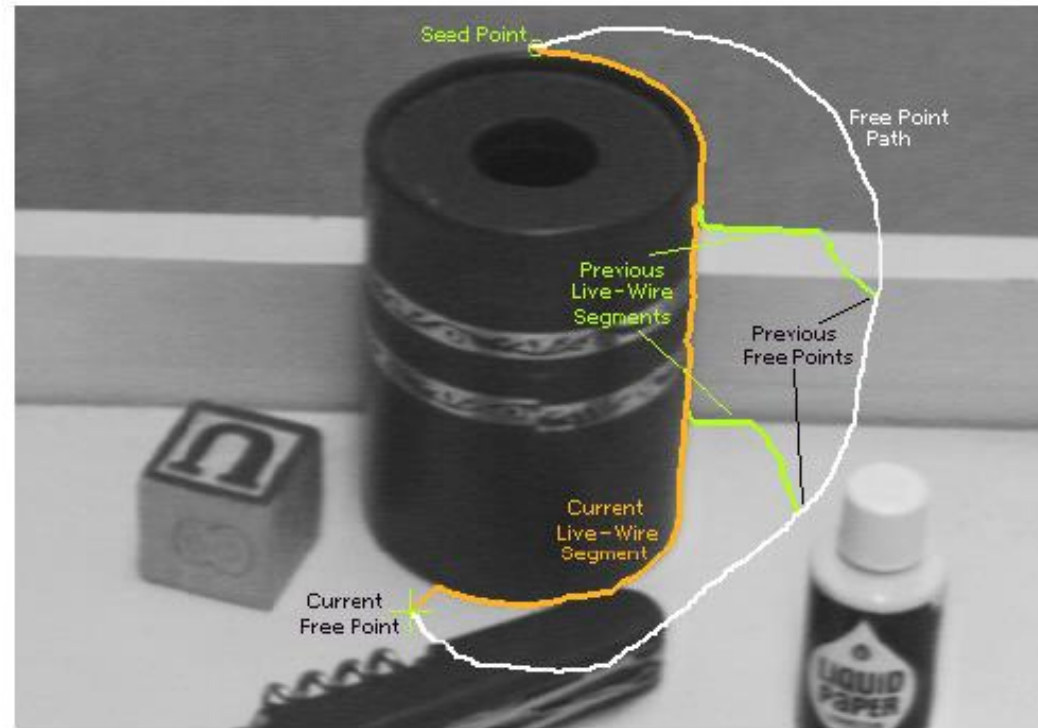


Figure 2: Image demonstrating how the live-wire segment adapts and snaps to an object boundary as the free point moves (via cursor movement). The path of the free point is shown in white. Live-wire segments from previous free point positions (t_0 , t_1 , and t_2) are shown in green.

boundary $\Rightarrow$ sharp interesting transitions.
usually straight lines are low cost but we need to make
the image edge low cost. Then curved edges will be
detected

Intelligent Scissors

- Approach answers a basic question
 - Q: how to find a path from seed to mouse that follows object boundary as closely as possible?
 - A: define a path that stays as close as possible to edges

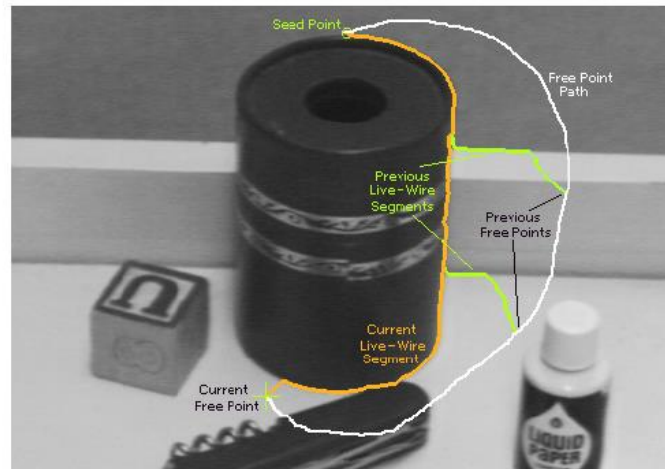
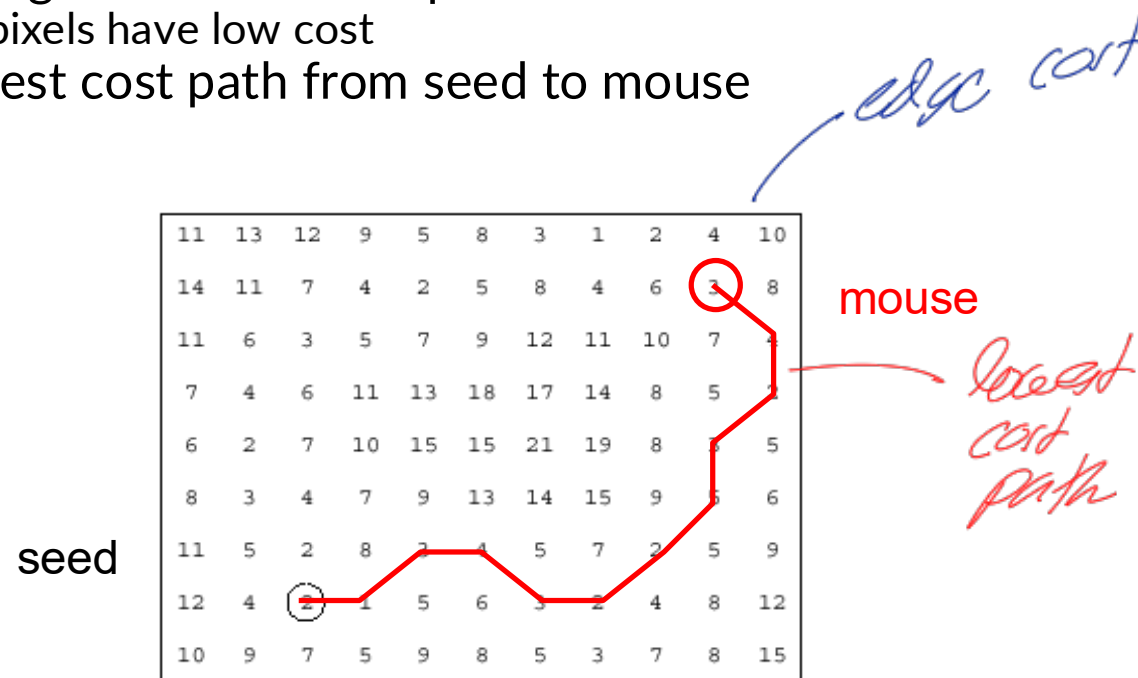


Figure 2: Image demonstrating how the live-wire segment adapts and snaps to an object boundary as the free point moves (via cursor movement). The path of the free point is shown in white. Live-wire segments from previous free point positions (t_0 , t_1 , and t_2) are shown in green.

Intelligent Scissors

- Basic Idea
 - Define edge score for each pixel
 - edge pixels have low cost
 - Find lowest cost path from seed to mouse



Questions

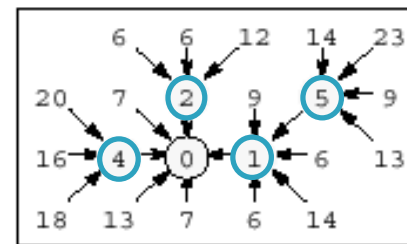
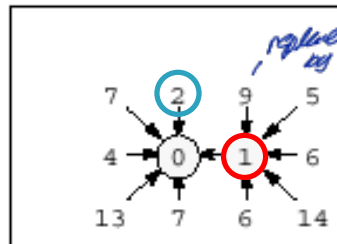
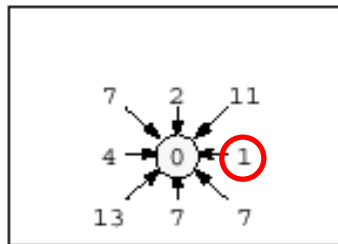
- How to define costs?
- How to find the path?

Path Search (Basic Idea)

- Graph Search Algorithm
 - Computes minimum cost path from seed to *all other pixels*

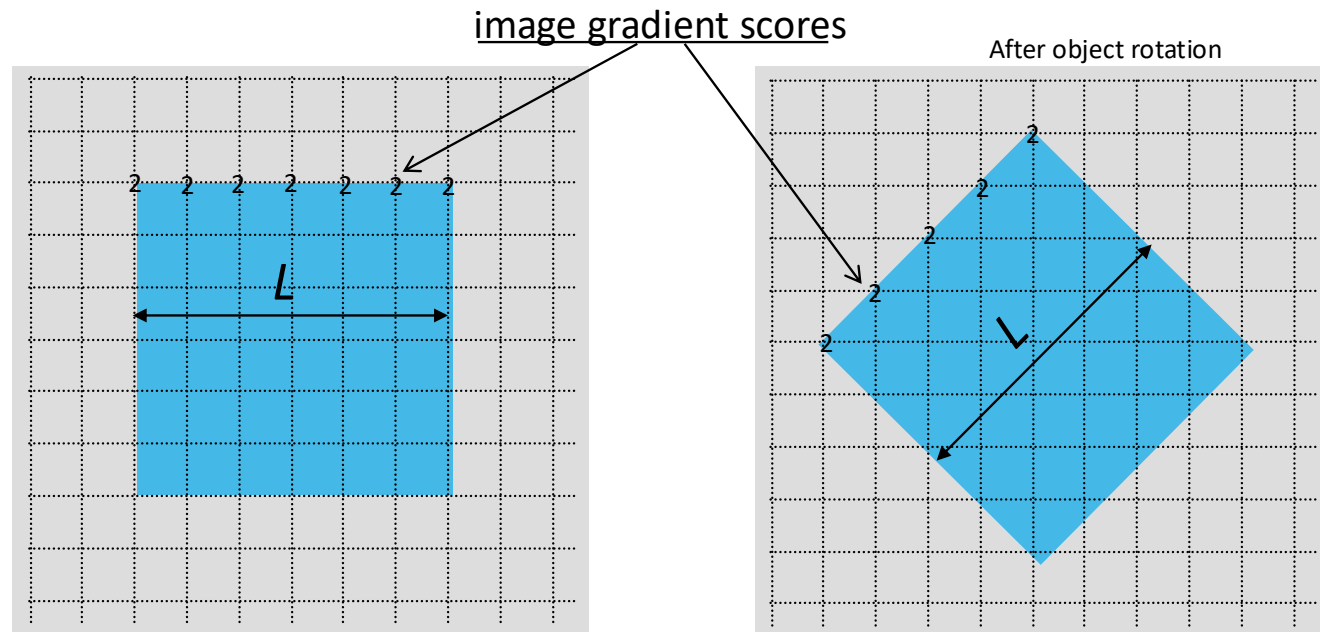
11	13	12	9	5	8	3	1	2	4	10
14	11	7	4	2	5	8	4	6	3	8
11	6	3	5	7	9	12	11	10	7	4
7	4	6	11	13	18	17	14	8	5	2
6	2	7	10	15	15	21	19	8	3	5
8	3	4	7	9	13	14	15	9	5	6
11	5	2	8	3	4	5	7	2	5	9
12	4	2	1	5	6	3	2	4	8	12
10	9	7	5	9	8	5	3	7	8	15

4
 ②
 being at 2 cost 0
 going up cost 4



Note: diagonal "paths" are scaled by a factor $\sqrt{2}$... **Why?**

A good segmentation method should be **Invariant to Image Rotation**



Path's cost along the top boundary

$$2 * 7 = 14$$

$\neq$

Path's cost along the top-left boundary

$$2 * 5 = 10$$

After diagonal links are adjusted by $\sqrt{2}$

$$2 * \sqrt{2} * 5 = 14$$

Scores Directly to Graph Edges

- Instead of nodes, image gradient scores directly to graph edges
- Treat the image as a graph

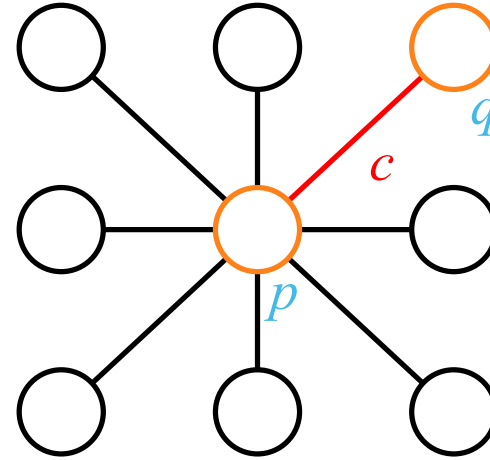
edge cut $\Rightarrow$ hard or soft cut.

- Graph

- node for every pixel p
- link between every adjacent pair of pixels, p, q
- cost c for each link

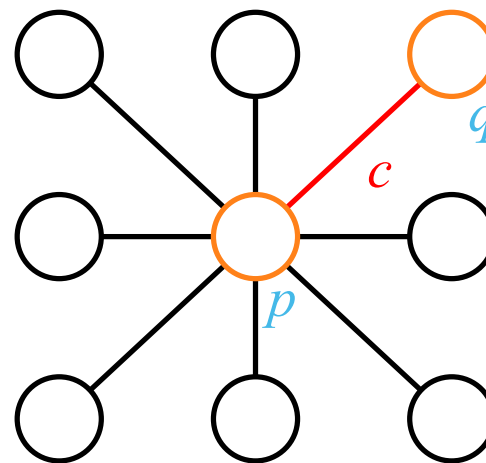
- Note: each *link* has a cost

- this is a little different than the figure before where each pixel (graph node) had a cost



Defining the Costs

- Treat the image as a graph
- Want to hug image edges:
how to define cost of a link?

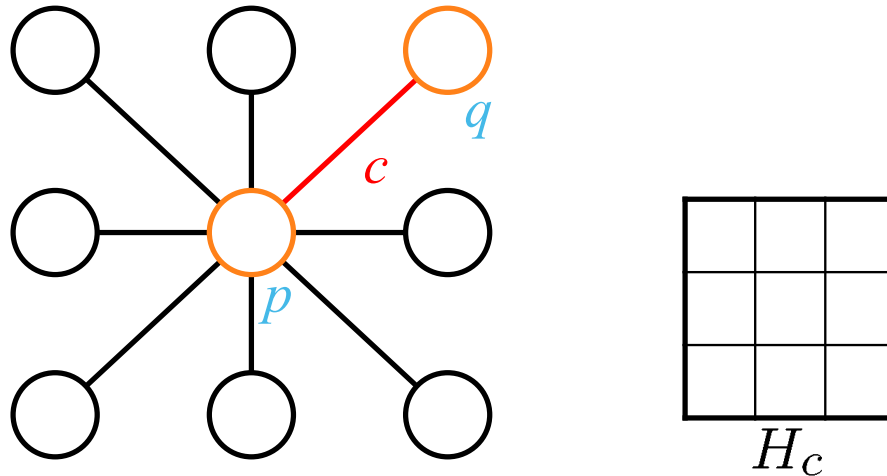


- the link should follow the intensity edge
 - want intensity to change rapidly $\perp$ to the link
- $c \approx - |\text{difference of intensity } \perp \text{ to the link}|$

high difference $\rightarrow$ high gradient
so we have low cost

Black
white

Defining the Costs

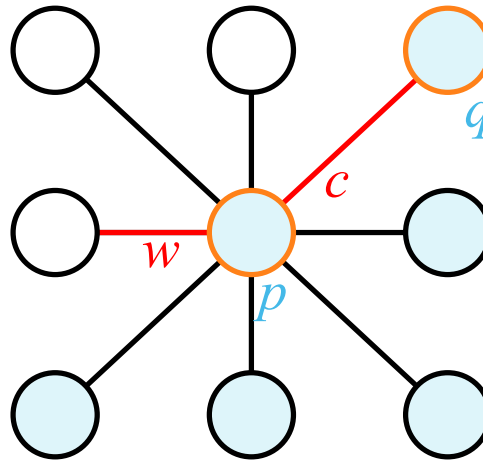


- c can be computed using a cross-correlation filter
 - assume it is centered at p
- Also typically scale c by its length
 - set $c = (\max - |\text{filter response}|)$
 - where $\max$ = maximum $|\text{filter response}|$ over all pixels in the image

Defining the Costs

The formulae used
to find gradient
edges

$$\frac{1}{4} \begin{bmatrix} 1 & 1 & \\ & & \\ -1 & -1 & \end{bmatrix} H_w$$



Why no $\sqrt{2}$ scalar?

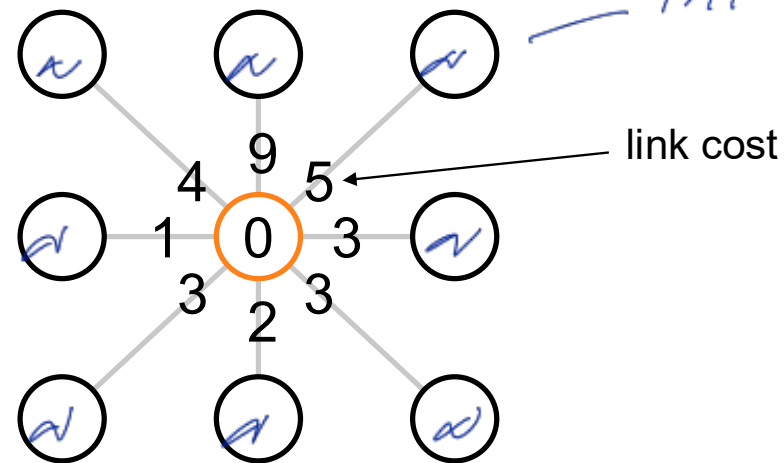
$$\begin{bmatrix} & 1 & \\ & & \\ & & -1 \end{bmatrix} H_c$$

need to $\frac{1}{\sqrt{2}}$
but also since
diagonal multiply
by $\sqrt{2}$
so cancel out.

- c can be computed using a cross-correlation filter
 - assume it is centered at p
- Also typically scale c by its length
 - set $c = (\max - |\text{filter response}|)$
 - where max = maximum |filter response| over all pixels in the image

(to get
difference)

Dijkstra's Shortest Path Algorithm



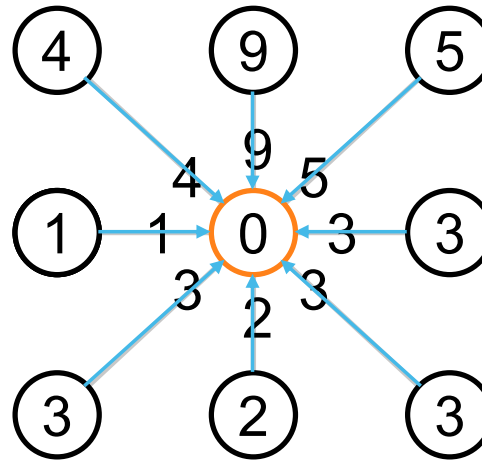
1. init node costs to ∞ , set p = seed point, $\text{cost}(p) = 0$
2. expand p as follows:
 - for each of p 's neighbors q that are not expanded
 - set $\text{cost}(q) = \min(\text{cost}(p) + c_{pq}, \text{cost}(q))$

$$\min(0 + c_{pq}, \infty)$$

↑
some inverse measure
of gradient.

ALGORITHM

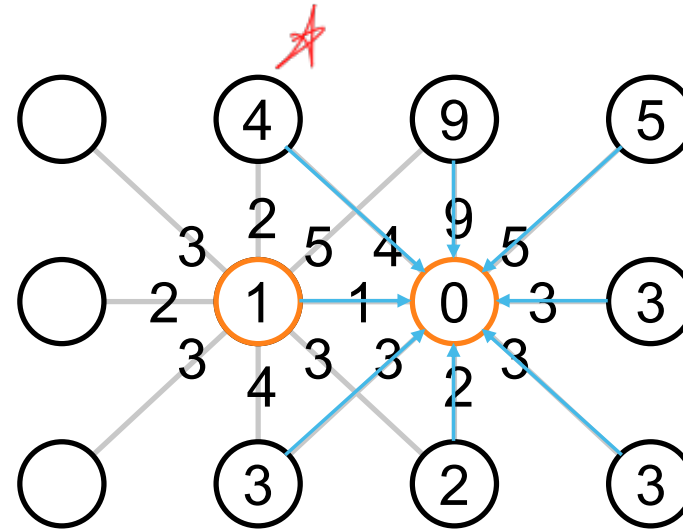
Dijkstra's Shortest Path Algorithm



1. init node costs to ∞ , set p = seed point, $\text{cost}(p) = 0$
2. expand p as follows:
 - for each of p 's neighbors q that are not expanded
 - set $\text{cost}(q) = \min(\text{cost}(p) + c_{pq}, \text{cost}(q))$
 - if q 's cost changed, make q point back to p
 - put q on the ACTIVE list (if not already there)

ALGORITHM

Dijkstra's Shortest Path Algorithm

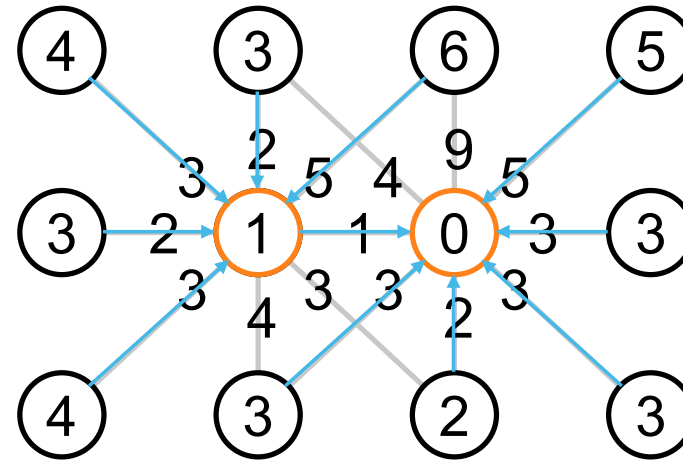


1. init node costs to ∞ , set p = seed point, $\text{cost}(p) = 0$
2. expand p as follows:
 - for each of p 's neighbors q that are not expanded
 - set $\text{cost}(q) = \min(\text{cost}(p) + c_{pq}, \text{cost}(q)) = 3$
 - if q 's cost changed, make q point back to p
 - put q on the ACTIVE list (if not already there)
3. set r = node with minimum cost on the ACTIVE list
4. repeat Step 2 for $p = r$

ALGORITHM

but need to remember path.

Dijkstra's Shortest Path Algorithm

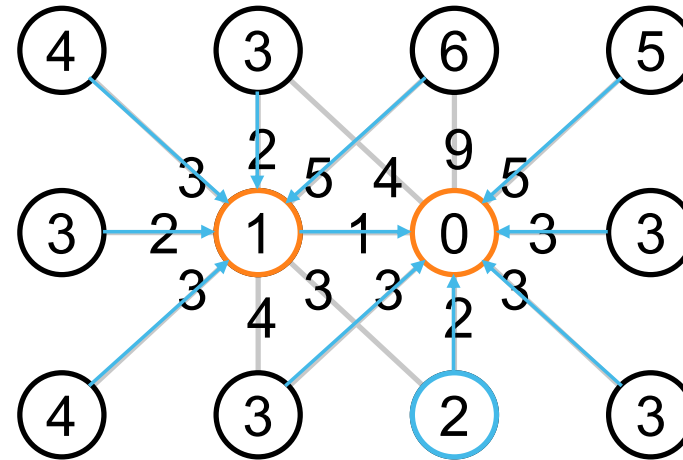


ALGORITHM

1. init node costs to ∞ , set p = seed point, $\text{cost}(p) = 0$
2. expand p as follows:
 - for each of p 's neighbors q that are not expanded
 - set $\text{cost}(q) = \min(\text{cost}(p) + c_{pq}, \text{cost}(q))$
 - if q 's cost changed, make q point back to p
 - put q on the ACTIVE list (if not already there)
3. set r = node with minimum cost on the ACTIVE list
4. repeat Step 2 for $p = r$

*use priority queues.
to get min value.*

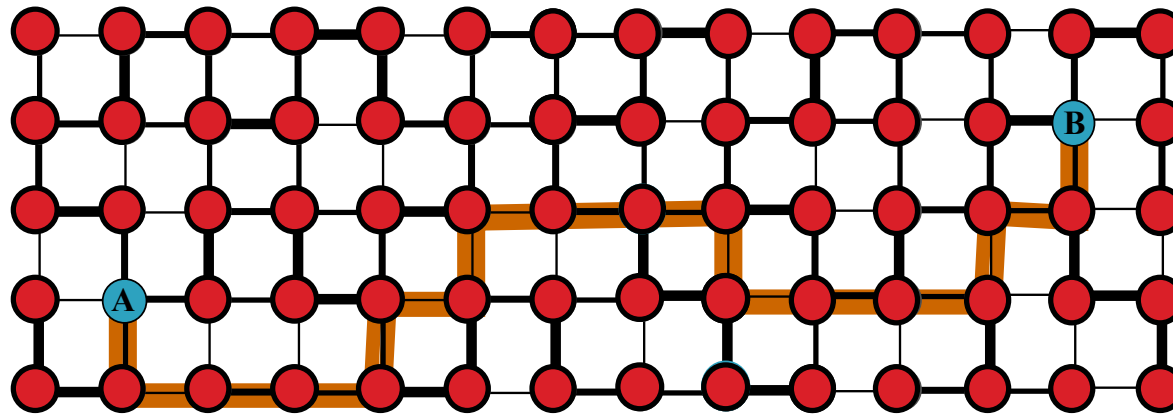
Dijkstra's Shortest Path Algorithm



1. init node costs to ∞ , set p = seed point, $\text{cost}(p) = 0$
2. expand p as follows:
 - for each of p 's neighbors q that are not expanded
 - set $\text{cost}(q) = \min(\text{cost}(p) + c_{pq}, \text{cost}(q))$
 - if q 's cost changed, make q point back to p
 - put q on the ACTIVE list (if not already there)
3. set r = node with minimum cost on the ACTIVE list
4. repeat Step 2 for $p = r$

ALGORITHM

Dijkstra's Shortest Path Algorithm



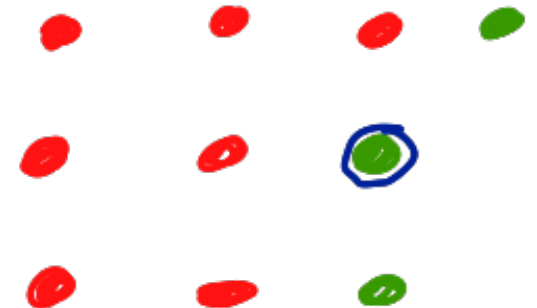
- - processed nodes (distance to A is known)
- - active nodes (front)
- - active node with the smallest distance value

stack

priority queue

Dijkstra algorithm

*Dijkstra takes me time
but backtracking takes no
time at all*



Dijkstra's Shortest Path Algorithm

- Properties
 - It computes the minimum cost path from the seed to every node in the graph. This set of minimum paths is represented as a *tree*
 - Running time, with N pixels:
 - $O(N^2)$ time if you use an active list
 - $O(N \log N)$ if you use an active priority queue (heap)
 - takes < second for a typical (640x480) image
 - Once this tree is computed once, we can extract the optimal path from any point to the seed in $O(N)$ time.
 - it runs in real time as the mouse moves
 - What happens when the user specifies a new seed?

Livewire Extensions

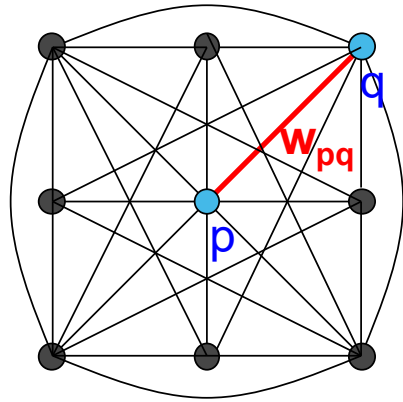
- Directed graphs
- Restricted search space
 - Restricted domain (e.g. near *a priori* model)
 - Restricted backward search
- Different edge weight functions
 - Image-Edge strength
 - Image-Edge Curvature
 - Proximity to known approximate model/boundary
- Multi-resolution processing





<http://www.cs.washington.edu/education/courses/cse455/>

Images as Graphs



Skipped
from
here

- *Fully-connected* graph
 - node (vertex) for every pixel
 - link between every pair of pixels, p, q
 - affinity weight w_{pq} for each link (edge)
 - w_{pq} measures *similarity*
 - similarity is *inversely proportional* to difference (in color and position...)

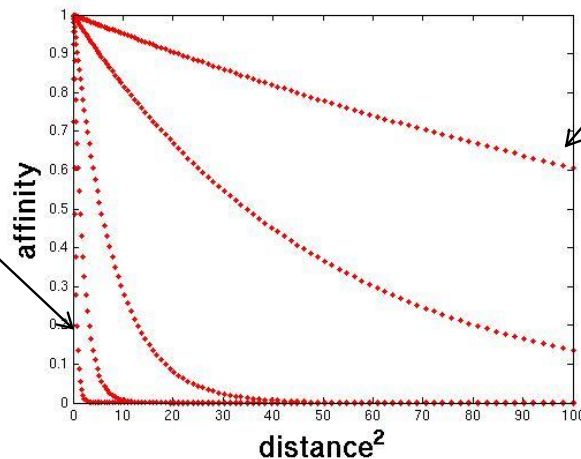
Source: Steve Seitz

Measuring Affinity

- One possibility:

$$aff(\mathbf{x}_i, \mathbf{x}_j) = \exp\left[-\frac{1}{2\sigma^2} \text{dist}(\mathbf{x}_i, \mathbf{x}_j)^2\right]$$

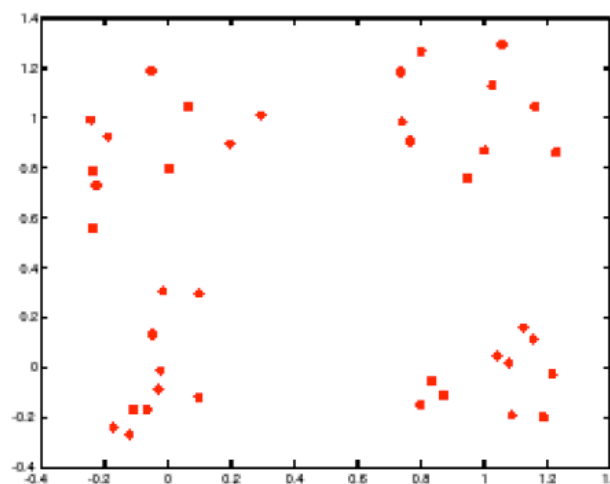
Small sigma:
group only
nearby points



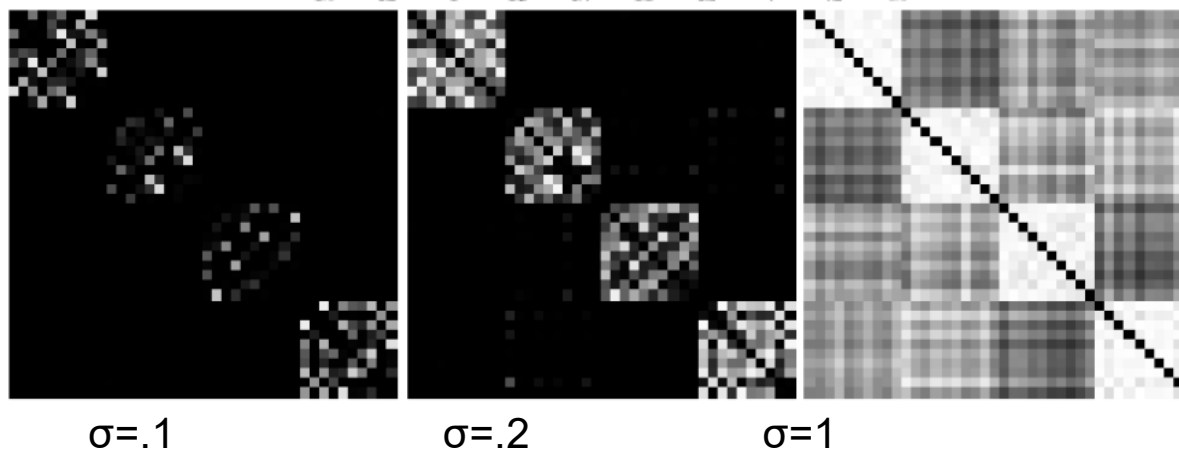
Large sigma:
group distant
points

Measuring Affinity

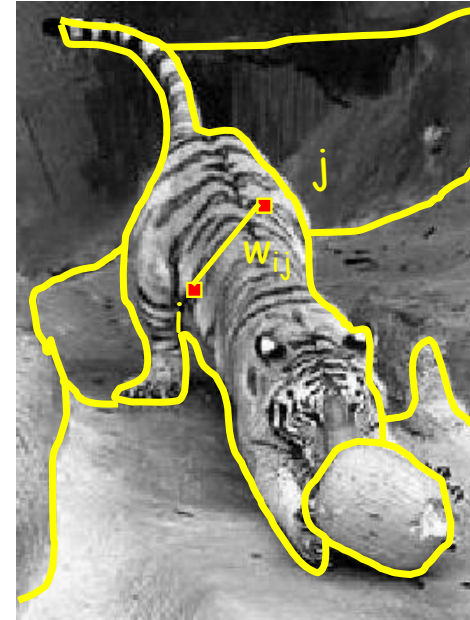
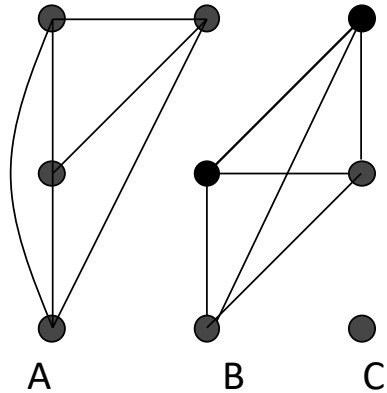
Data points



Affinity
matrices



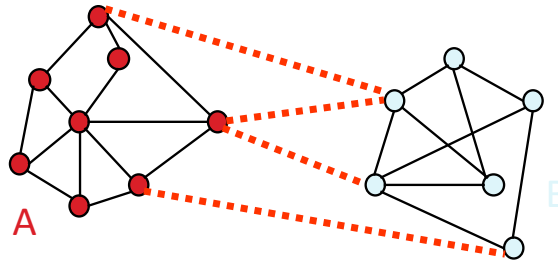
Segmentation by Graph Partitioning



- Break Graph into Segments
 - Delete links that cross between segments
 - Easiest to break links that have low affinity
 - similar pixels should be in the same segments
 - dissimilar pixels should be in different segments

Source: S. Seitz

Graph Cut



- Set of edges whose removal makes a graph disconnected
- Cost of a cut: sum of weights of cut edges

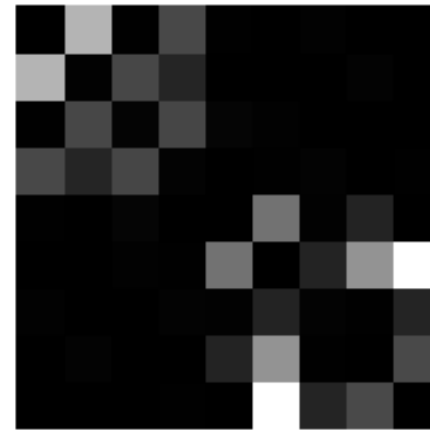
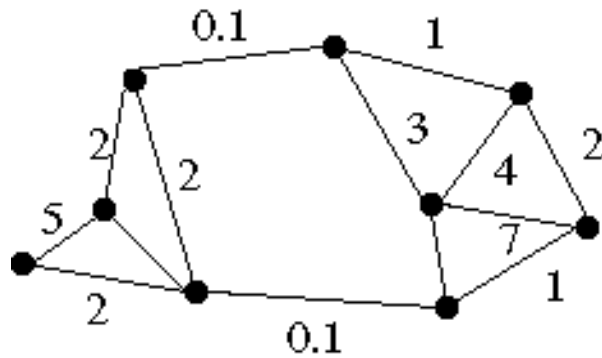
$$cut(A, B) = \sum_{p \in A, q \in B} w_{p,q}$$

- A graph cut gives us a segmentation
 - What is a “good” graph cut and how do we find one?

Minimum Cut

- We can do segmentation by finding the *minimum cut* in a graph
 - Efficient algorithms exist for doing this

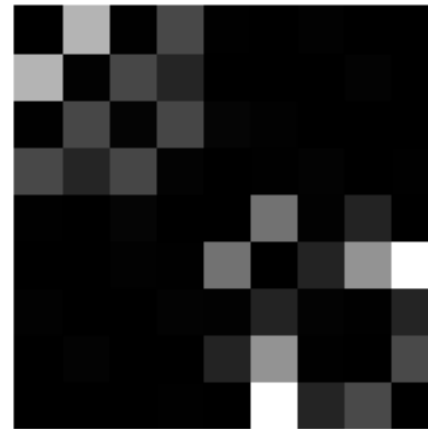
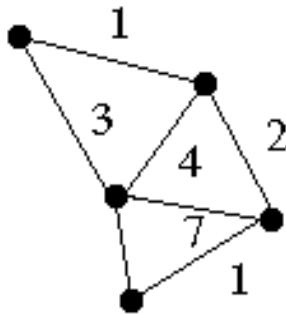
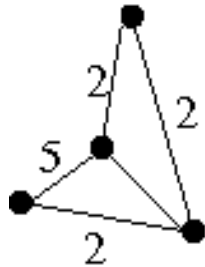
Minimum cut example



Minimum Cut

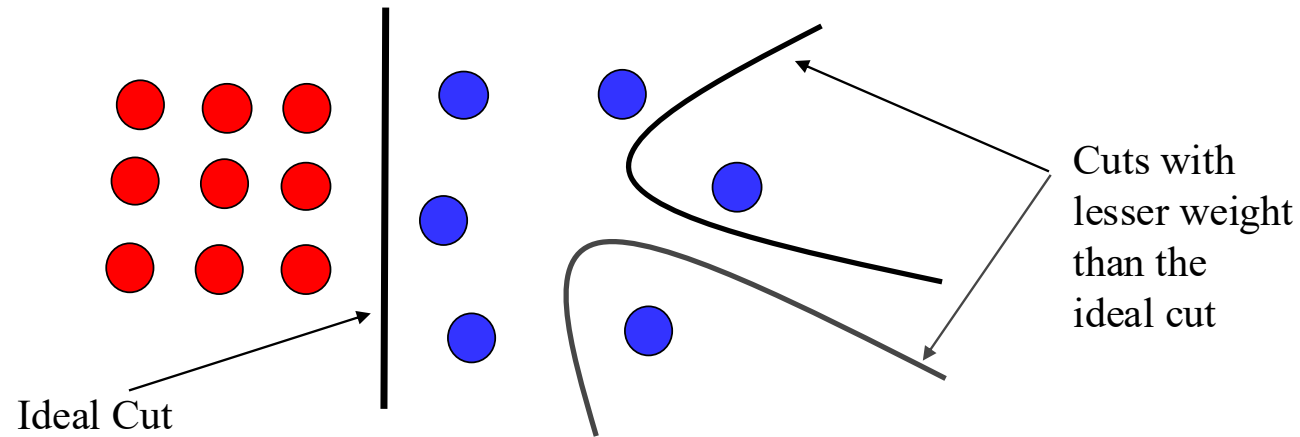
- We can do segmentation by finding the *minimum cut* in a graph
 - Efficient algorithms exist for doing this

Minimum cut example

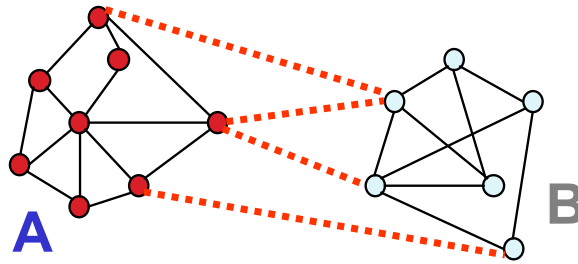


Minimum Cut

- Drawback: minimum cut tends to cut off very small, isolated components



Cuts in a Graph: Normalized Cut



Normalized Cut

- Fixes bias of Min Cut by **normalizing** for size of segments:

$$Ncut(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)}$$

$assoc(A, V)$ = sum of weights of all edges that touch A

- Ncut value small when we get two clusters with many edges with high weights, and few edges of low weight between them
- Approximate solution for minimizing the Ncut value : generalized eigenvalue problem.

Normalized Cut

- Let W be the adjacency matrix of the graph
- Let D be the diagonal matrix with diagonal entries $D(i, i) = \sum_j W(i, j)$
- Then the normalized cut cost can be written as

$$\frac{y^T (D - W) y}{y^T D y}$$

- where y is an indicator vector whose value should be 1 in the i th position if the i th feature point belongs to A and a negative constant otherwise

Normalized Cut

- Finding the exact minimum of the normalized cut cost is NP-complete, but if we *relax* y to take on arbitrary values, then we can minimize the relaxed cost by solving the *generalized eigenvalue problem* $(D - W)y = \lambda Dy$
- The solution y is given by the generalized eigenvector corresponding to the second smallest eigenvalue
- Intuitively, the i th entry of y can be viewed as a “soft” indication of the component membership of the i th feature
 - Can use 0 or median value of the entries as the splitting point (threshold), or find threshold that minimizes the Ncut cost

Normalized Cut Algorithm

1. Represent the image as a weighted graph
 $G = (V, E)$, compute the weight of each edge, and summarize the information in D and W
 2. Solve $(D - W)y = \lambda Dy$ for the eigenvector with the second smallest eigenvalue
 3. Use the entries of the eigenvector to bipartition the graph
- To find more than two clusters:
 - Recursively bipartition the graph
 - Run k-means clustering on values of several eigenvectors

Example Results



Results: Berkeley Segmentation Engine



<http://www.cs.berkeley.edu/~fowlkes/BSE/>

Normalized Cuts: Pros and Cons

Pros:

- Generic framework, flexible to choice of function that computes weights (“affinities”) between nodes
- Does not require model of the data distribution

Cons:

- Time complexity can be high
 - Dense, highly connected graphs → many affinity computations
 - Solving eigenvalue problem
- Preference for balanced partitions

SNAKES

Deformable Models in 2D

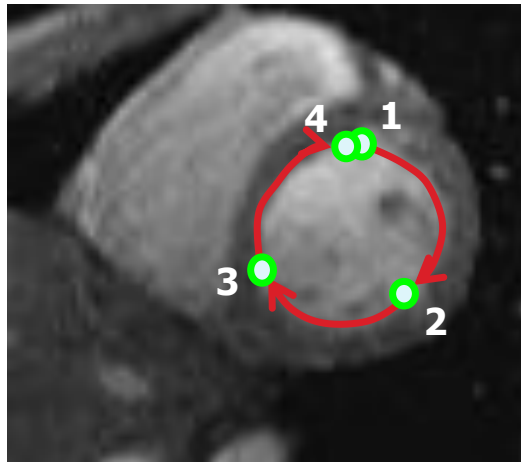
- Active Contours or “snakes”
 - “snakes” vs. “livewire”
 - (discrete) energy formulations for snakes
 - relation to Hidden Markov Models (HMM)
- Optimization (discrete case)
 - Gradient Descent
 - Dynamic Programming (DP), Viterbi algorithm
 - DP versus Dijkstra

Extra Reading: Sonka et.al 5.2.5 and 8.2

Active Contours by Blake and Isard

“Live-wire” vs. “Snakes”

- intelligent scissors [Mortensen, Barrett 1995]
- live-wire [Falcao, Udupa, Samarasekera, Sharma 1998]



Shortest paths on image-based graph connect seeds placed on object boundary

“Live-wire” vs. “Snakes”

- Snakes, active contours [Kass, Witkin, Terzopoulos 1987]
- In general, *deformable models* are widely used



Given: initial contour (model) near desirable object

“Live-wire” vs. “Snakes”

- Snakes, active contours [Kass, Witkin, Terzopoulos 1987]
- In general, *deformable models* are widely used



Given: initial contour (model) near desirable object

Goal: evolve the contour to fit exact object boundary

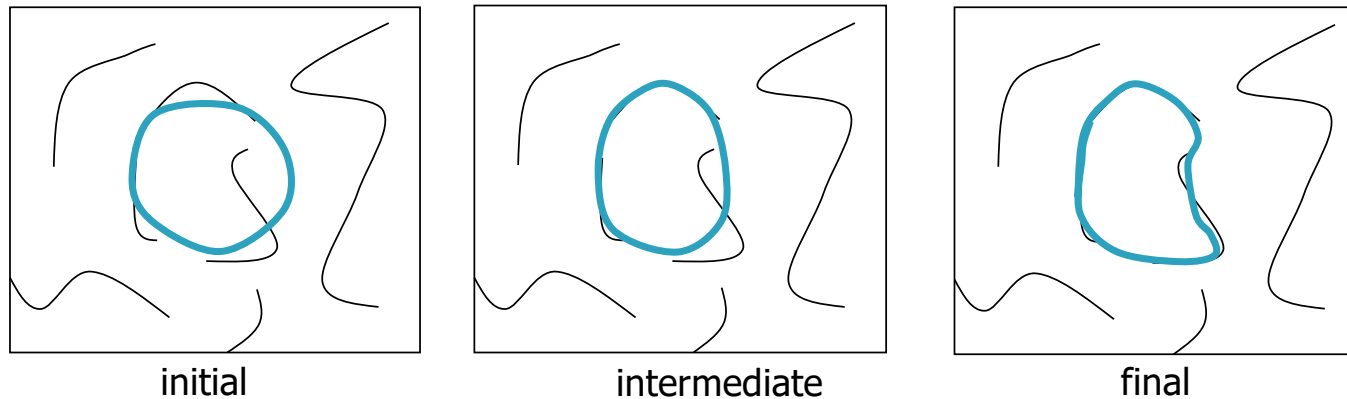
Tracking via Deformable Models



Tracking Heart Ventricles

“Snakes”

- A smooth 2D curve which matches to image data
- Initialized near target, iteratively refined
- Can restore missing data



Q: How does that work?

Snake Energy

The total energy of snake is defined as

$$E_{total} = E_{in} + E_{ex}$$

Internal energy encourages
smoothness or any particular shape

Internal energy incorporates prior
knowledge about object boundary
allowing to extract boundary even if
some image data is missing

External energy encourages curve onto
image structures (e.g. image edges)

Q: Is there any energy behind 1) image thresholding
2) region growing
3) live-wire

NO

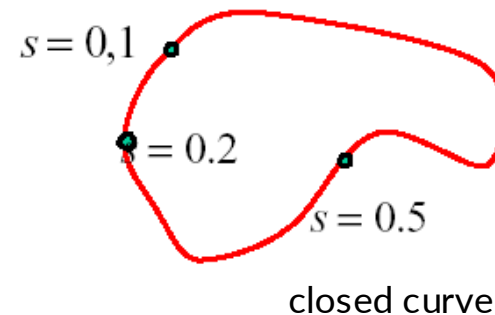
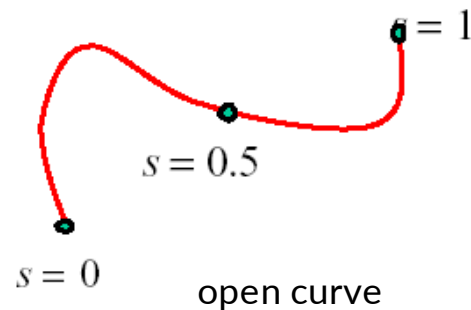
NO

YES (will compare later)

Parametric Curve Representation (continuous case)

- A curve can be represented parametrically

$$\nu(s) = (x(s), y(s)) \quad 0 \leq s \leq 1$$



Note: in computer vision and medical image analysis communities the term “snake” is normally associated with such *parametric representation of contours*. There are other Ways to represent contours/surfaces! (Will talk about that later)

Internal Energy

- The bending energy of a continuous curve is

$$E_{in}(v(s)) = \underbrace{\alpha(s) \left| \frac{dv}{ds} \right|^2}_{\text{Elasticity}} + \underbrace{\beta(s) \left| \frac{d^2v}{ds^2} \right|^2}_{\text{Stiffness}}$$

The more the curve bends, the larger this value

$$E_{in} = \int_0^1 E_{in}(v(s)) ds$$

External energy

- The external energy describes how well the curve matches the image data locally
- Numerous forms can be used, attracting the curve toward different image features

Simple Edge Strength

- Suppose we have an image $I(x,y)$
- Generate gradient images $G_x(x,y)$ & $G_y(x,y)$
- External energy based on edge strength at a point is then

$$G_x(x,y) \quad G_y(x,y)$$

- An external energy term for a snake is thus

$$E_{ex}(\nu(s)) = -(|G_x(\nu(s))|^2 + |G_y(\nu(s))|^2)$$

(Negative in order that minimizing it forces the curve toward large edges)

$$E_{ex} = \int_0^1 E_{ex}(\nu(s)) ds$$

Snake Energy (Continuous Form)

- The total energy of the snake is defined as

$$E_{total} = E_{in} + E_{ex}$$

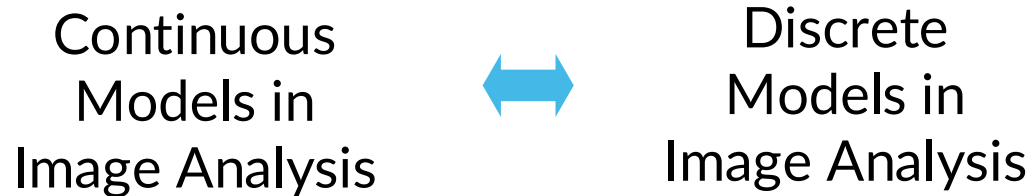
$$E_{in} = \int_0^1 E_{in}(\nu(s)) ds$$

e.g. bending energy

$$E_{ex} = \int_0^1 E_{ex}(\nu(s)) ds$$

e.g. total edge strength
under curve

Discrete and Continuous approaches



The relationship is somewhat analogous to relation between continuous and discrete models in physics, e.g.



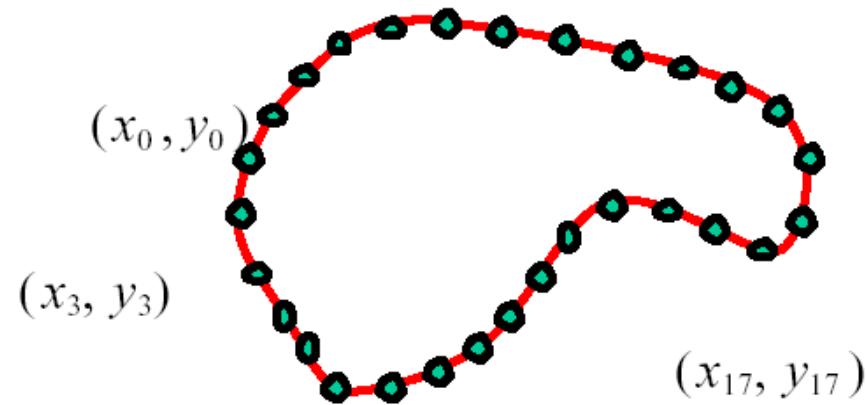
Most of the actual properties of light can be explained either way.

In this course we concentrate on discrete models in image analysis.
In particular, discrete models can be directly optimized by combinatorial algorithms which are a good match for naturally “discrete” computers.

Parametric Curve Representation (discrete case)

- Represent the curve with a set of n point

$$\mathbf{v}_i = (x_i, y_i) \quad i = 0 \dots n-1$$



Discrete Representation

- If the curve is represented by n points

$$\mathbf{v}_i = (x_i, y_i) \quad i = 0 \dots n-1$$

$$\frac{d\mathbf{v}}{ds} \approx \frac{\mathbf{v}_{i+1} - \mathbf{v}_{i-1}}{2} \quad \frac{d^2\mathbf{v}}{ds^2} \approx (\mathbf{v}_{i+1} - \mathbf{v}_i) - (\mathbf{v}_i - \mathbf{v}_{i-1}) = \mathbf{v}_{i+1} - 2\mathbf{v}_i + \mathbf{v}_{i-1}$$

$$E_{in} = \sum_{i=0}^{n-1} \boxed{\alpha \left| \mathbf{v}_{i+1} - \mathbf{v}_i \right|^2} + \boxed{\beta \left| \mathbf{v}_{i+1} - 2\mathbf{v}_i + \mathbf{v}_{i-1} \right|^2}$$

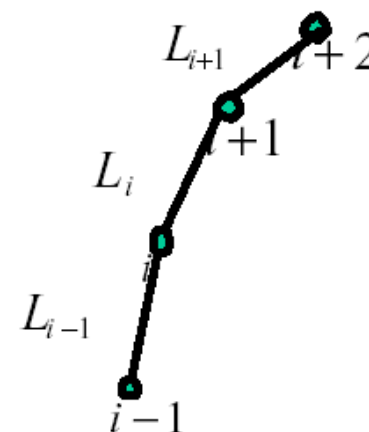
Elasticity

Stiffness

Simple Elastic Curve

- For a curve represented as a set of points a simple elastic energy term is

$$\begin{aligned} E_{in} &= \alpha \cdot \sum_{i=0}^{n-1} L_i^2 \\ &= \alpha \cdot \sum_{i=0}^{n-1} (x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2 \end{aligned}$$



This encourages the closed curve to shrink to a point (like a very small elastic band)

Simple Edge Strength

- An external energy term for a (discrete) snake based on image edge

$$E_{ex} = - \sum_{i=0}^{n-1} |G_x(x_i, y_i)|^2 + |G_y(x_i, y_i)|^2$$

(Negative in order that minimizing it forces the curve toward large edges)

Simple Elastic Snake

- A simple elastic snake is thus defined by
 - A set of n points,
 - An internal elastic energy term
 - An external edge based energy term
- To use this to locate the outline of an object
 - Initialize in the vicinity of the object
 - Modify the points to minimize the total energy

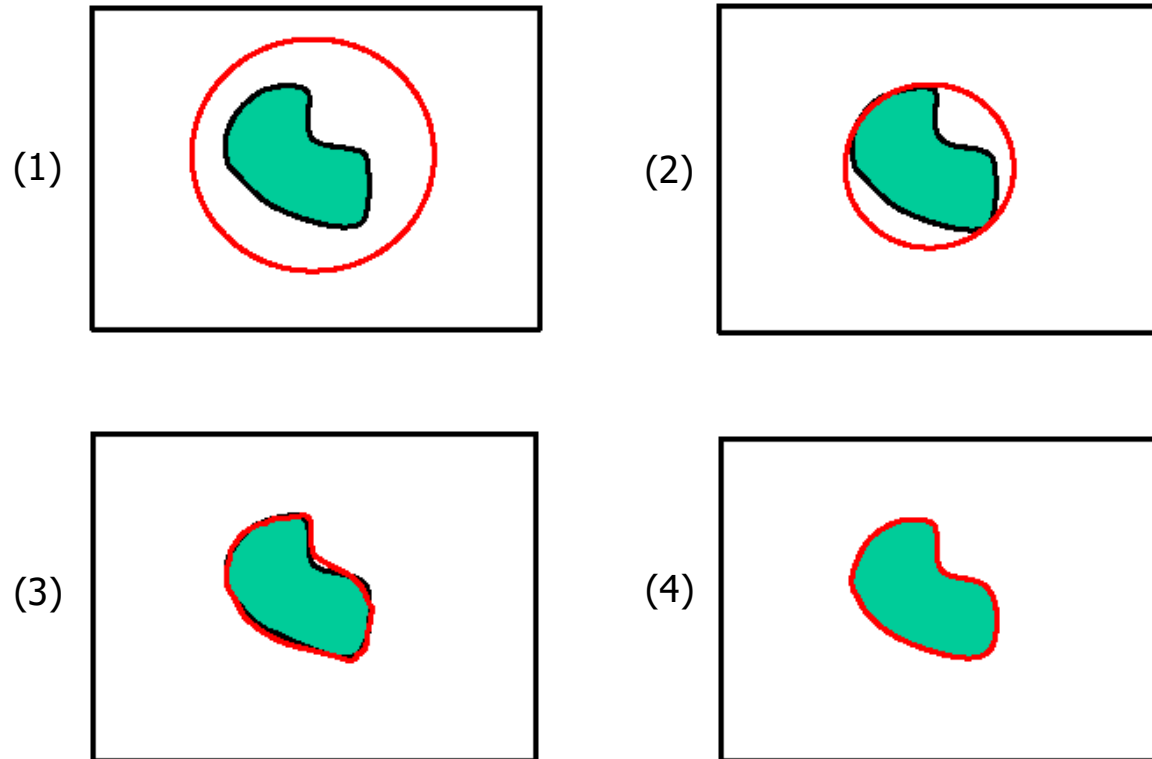
Simple Elastic Snake

- In this case we have

$$E_{total}(x) = E_{in}(x) + E_{ex}(x)$$

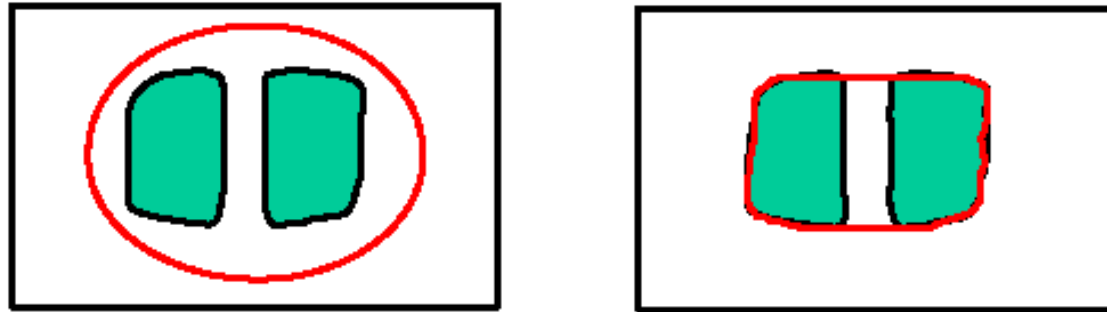
- where $x = (x_0, \dots, x_{n-1}, y_0, \dots, y_{n-1})^T$
- Optimization problem ($2n$ variables)
 - note that we can easily compute the derivatives, which allows efficient optimization (via **gradient descent**) converging to a local minima
 - more robust option: **Dynamic Programming** (later)

Synthetic Example



Dealing with Missing Data

- The smoothness constraint can deal with missing data:



Relative Weighting

- Notice that the strength of the internal elastic component can be controlled by a parameter, α .
- Increasing this increases stiffness of curve

$$E_{in} = \alpha \cdot \sum_{i=0}^{n-1} L_i^2$$



large α



medium α



small α

Encouraging point spacing

- To stop the curve from shrinking to a point

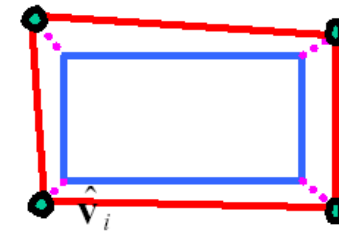
$$E_{in} = \alpha \cdot \sum_{i=0}^{n-1} (L_i - \hat{L}_i)^2$$

- encourages particular point separation

Simple Shape Prior

- If object is some smooth variation on a known shape, use

$$E_{in} = \alpha \cdot \sum_{i=0}^{n-1} (v_i - \hat{v}_i)^2$$



- where $\{\hat{v}_i\}$ give points of the basic shape

- May use a statistical (Gaussian) shape model

$$E_{in} = -\ln N(v | \hat{v}) \propto (v - \hat{v})^T \cdot C \cdot (v - \hat{v})$$

Alternative External (Image) Energies

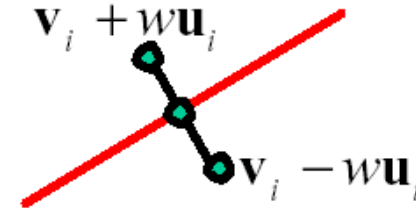
- Directed gradient measures

$$E_{ex} = - \sum_{i=0}^{n-1} \left| u_{x,i} G_x(v_i) + u_{y,i} G_y(v_i) \right|$$

- Where $u_i = (u_{x,i}, u_{y,i})$ is the unit normal to the boundary at contour point v_i
- This gives a good response when the boundary has the same direction as the edge, but weaker responses when it does not

Alternative External (Image) Energies

- Line strength measure



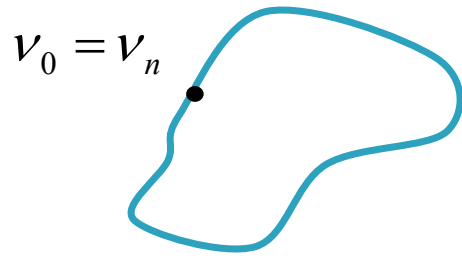
$$E_{ex} = - \sum_{i=0}^{n-1} I(v_i) - 0.5 [I(v_i + wu_i) + I(v_i - wu_i)]$$

- Where u_i is the unit normal at the point, w is an estimate of line half-width

ω

- Strong response when on a bright line
- Looks for white lines on dark background

Open and Closed Curves



closed curve



open curve

- When using an open curve we can impose constraints on the end points (e.g. end points may have fixed position)
 - **Q:** What are similarities and differences with the live-wire if the end points of an open snake are fixed?

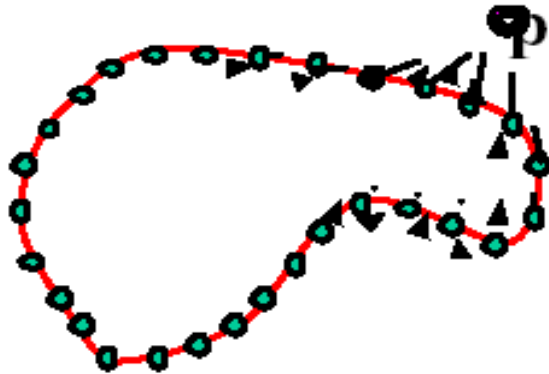
Additional Constraints

- Snakes originally developed for interactive image segmentation
- Initial snake result can be nudged where it goes wrong
- Simply modify the external energy term to
 - Pull nearby points toward cursor, or
 - Push nearby points away from cursor

Interactive (External) Forces

- Pull points towards cursor:

$$E_{pull} = - \sum_{i=0}^{n-1} \frac{r^2}{|v_i - p|^2}$$



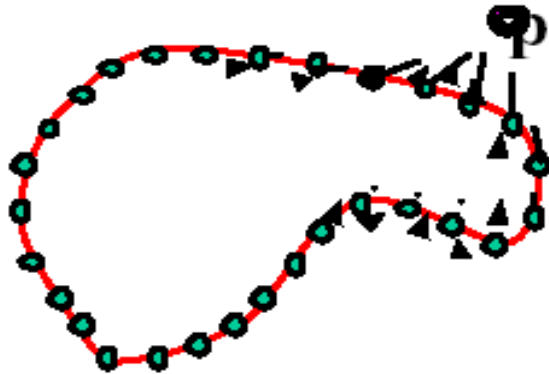
Nearby points get
pulled hardest

Negative sign gives
better energy for
positions near p

Interactive (External) Forces

- Push points from cursor:

$$E_{push} = + \sum_{i=0}^{n-1} \frac{r^2}{|v_i - p|^2}$$



Nearby points get pushed hardest

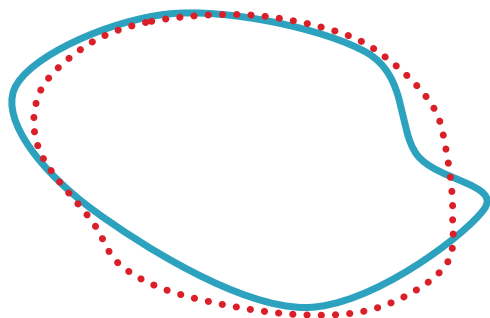
Positive sign gives better energy for positions far from p

Dynamic snakes

- Adding motion parameters into hidden variables (for each snake node)
- Introduce energy terms for motion consistency

Discrete Snakes Optimization

- At each iteration we compute a **new** snake position within proximity to the **previous** snake
- **New** snake energy should be smaller than the **previous** one
- Stop when the energy can not be decreased within local neighborhood of the snake (local energy minima)

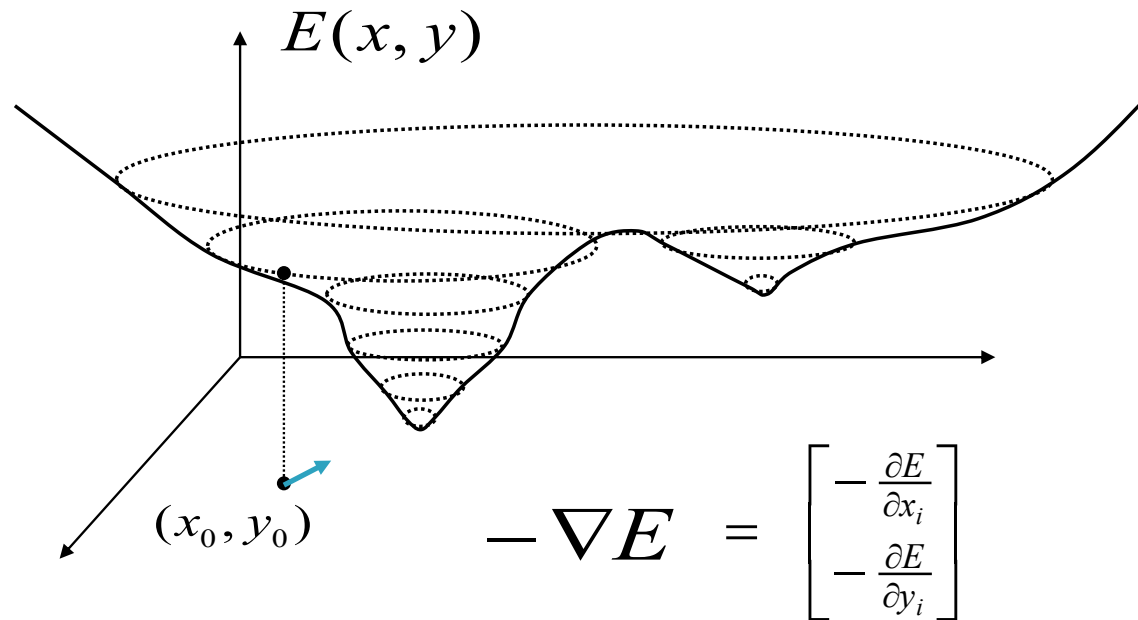


Optimization Methods

1. *Gradient Descent*
2. *Dynamic Programming*

Gradient Descent

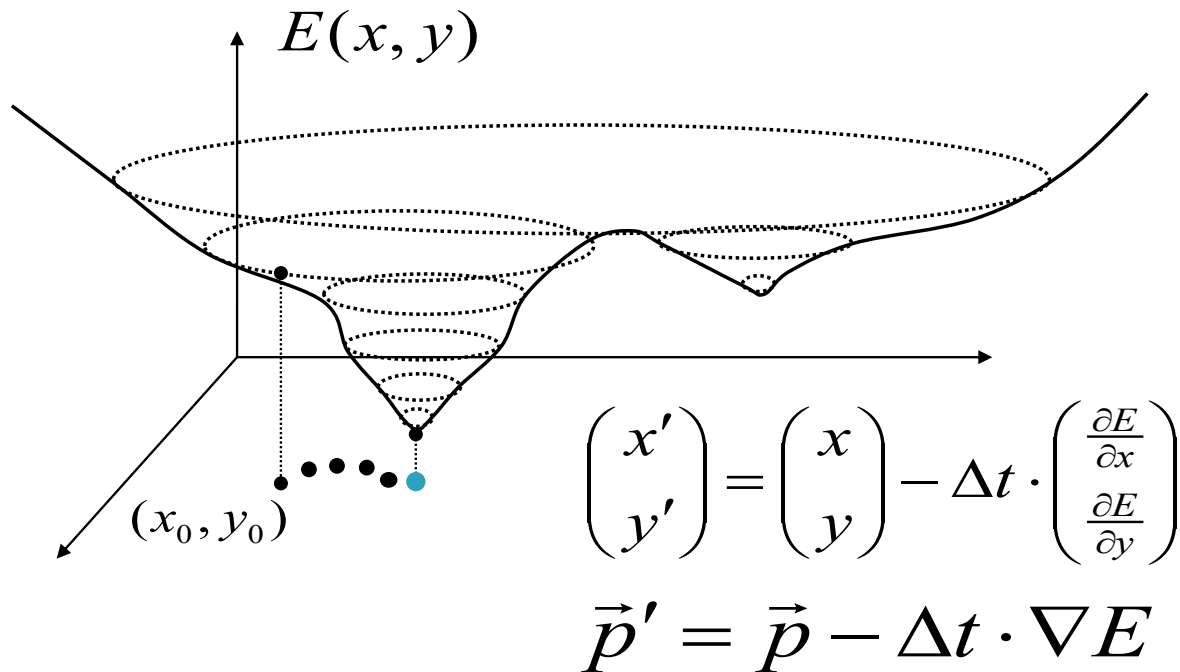
- Example: minimization of functions of 2 variables



negative gradient at point (x, y) gives direction of the steepest descent towards lower values of function E

Gradient Descent

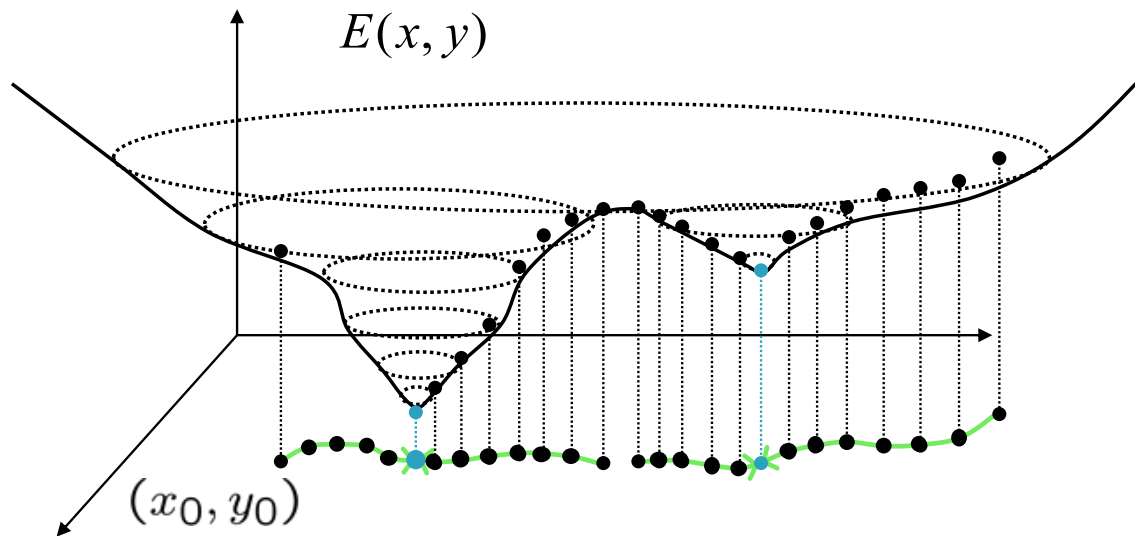
- Example: minimization of functions of 2 variables



Stop at a **local minima** where $\nabla E = \vec{0}$

Gradient Descent

- Example: minimization of functions of 2 variables



High sensitivity wrt. the initialisation !!

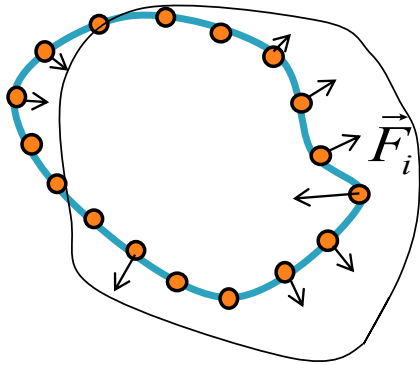
Gradient Descent for Snakes

simple elastic snake

$$E_{total}(x_0, \dots, x_{n-1}, y_0, \dots, y_{n-1}) = - \sum_{i=0}^{n-1} |G_x^{energy}(x_i, y_i)|^2 + |G_y(x_i, y_i)|^2$$

energy as a function 2n variables

$$+ \alpha \cdot \sum_{i=0}^{n-1} (x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2$$



$$\vec{F} = -\nabla E$$

Force is a negative gradient
of energy function
(2n-dimensional vector)

$$= \begin{bmatrix} \vec{F}_0 \\ \vec{F}_1 \\ \dots \\ \vec{F}_{n-1} \end{bmatrix}$$

We can break force/gradient into
components corresponding to
individual snake nodes

$$\vec{F}_i = - \begin{bmatrix} \frac{\partial E}{\partial x_i} \\ \frac{\partial E}{\partial y_i} \end{bmatrix}$$

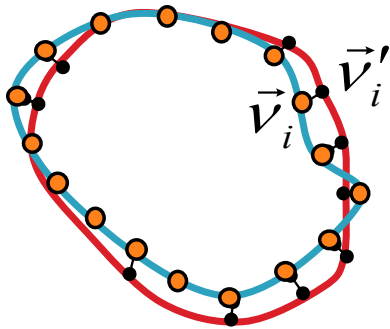
Discrete Snakes: “Gradient Flow” evolution

$$dC = -dt \cdot \nabla E$$

Contour evolution via
“Gradient flow”

Update equation for each node

$$\vec{v}'_i = \vec{v}_i + \underbrace{\vec{F}_i \cdot \Delta t}_{\Delta \vec{v}_i} \quad i = 0, \dots, n-1$$



Stopping criteria:

$$\vec{F}_i = -\nabla E|_i = 0 \quad \text{for all } i$$

at a local minima of energy

Difficulties with Gradient Descent

- Very difficult to obtain accurate estimates of high-order derivatives on images (discretization errors)
 - E.g., estimating $\frac{\partial E}{\partial x}$ requires computation of $\frac{\partial G_x}{\partial x} = \frac{\partial^2 I}{\partial x^2}$.
- Gradient descent is not trivial even for one-dimensional functions. Robust numerical performance for $2n$ -dimensional function is problematic.
 - Choice of parameter Δt is non-trivial
 - Small Δt , the algorithm may be too slow
 - Large Δt , the algorithm may never converge
 - Even if “converged” to a good local minima, the snake is likely to oscillate near it

Alternative Solution for 2D Snakes: Dynamic Programming

- In most cases, snake energy can be rewritten as a sum of pair-wise interaction potential

$$E_{total}(v_0, \dots, v_{n-1}) = \sum_{i=0}^{n-1} E_i(v_i, v_{i+1})$$

- More generally, it can be written as a sum of triple-interaction potentials.

$$E_{total}(v_0, \dots, v_{n-1}) = \sum_{i=0}^{n-1} E_i(v_{i-1}, v_i, v_{i+1})$$

Snake Energy: Pair-Wise Interactions

Example: simple elastic snake energy

$$E_{total}(x_0, \dots, x_{n-1}, y_0, \dots, y_{n-1}) = - \sum_{i=0}^{n-1} |G_x(x_i, y_i)|^2 + |G_y(x_i, y_i)|^2 \\ + \alpha \cdot \sum_{i=0}^{n-1} (x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2$$

$$E_{total}(v_0, \dots, v_{n-1}) = - \sum_{i=0}^{n-1} \|G(v_i)\|^2 + \alpha \cdot \sum_{i=0}^{n-1} \|v_{i+1} - v_i\|^2$$

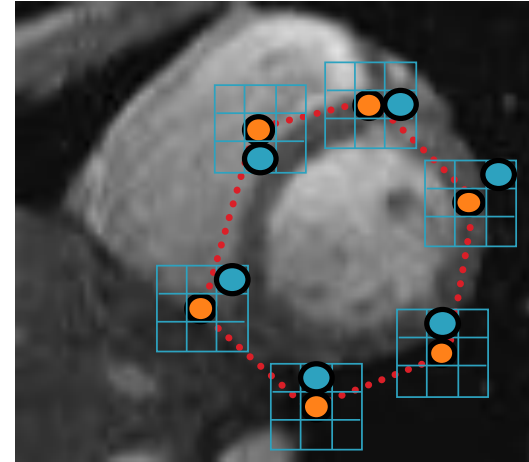
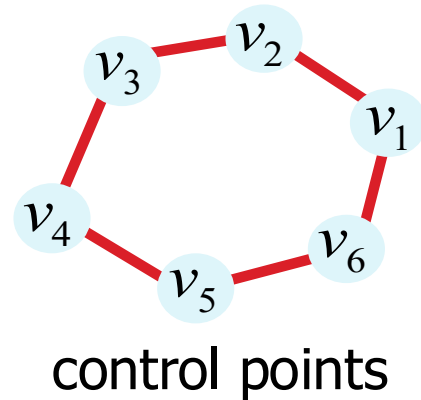
$$E_{total}(v_0, \dots, v_{n-1}) = \sum_{i=0}^{n-1} E_i(v_i, v_{i+1})$$

$$\text{where } E_i(v_i, v_{i+1}) = -\|G(v_i)\|^2 + \alpha \|v_i - v_{i+1}\|^2$$

Q: give an example of snake with triple-interaction potentials?

DP Snakes

[Amini, Weymouth, Jain, 1990]

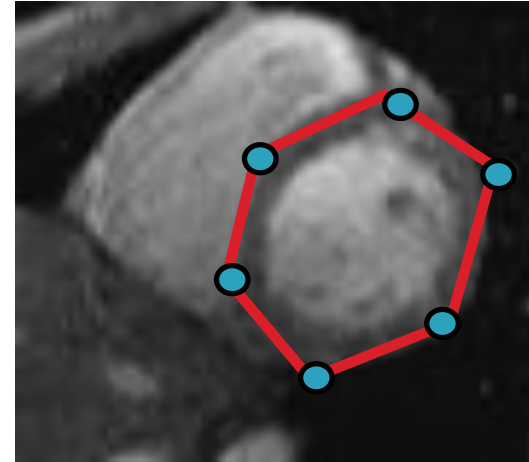
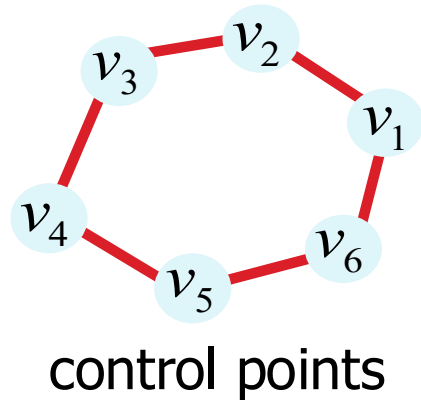


$$E(v_1, v_2, \dots, v_n) = E_1(v_1, v_2) + \overset{\text{First-order interactions}}{E_2(v_2, v_3)} + \dots + E_{n-1}(v_{n-1}, v_n)$$

Energy E is minimized via Dynamic Programming

DP Snakes

[Amini, Weymouth, Jain, 1990]



$$E(v_1, v_2, \dots, v_n) = E_1(v_1, v_2) + \overset{\text{First-order interactions}}{E_2(v_2, v_3)} + \dots + E_{n-1}(v_{n-1}, v_n)$$

Energy E is minimized via Dynamic Programming

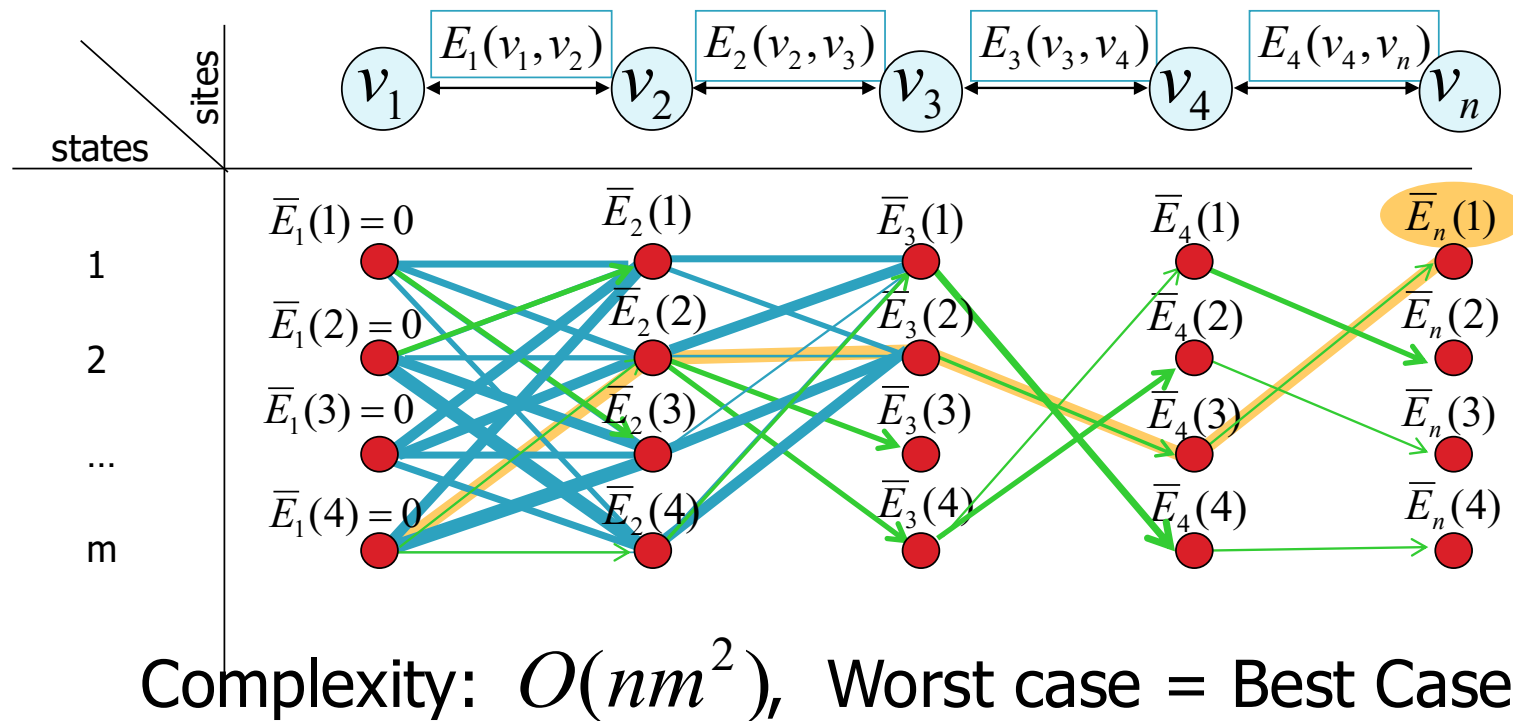
Iterate until optimal position for each point is the center of the box,
i.e. the **snake is optimal in the local search space** constrained by boxes

Dynamic Programming (DP)

Viterbi Algorithm

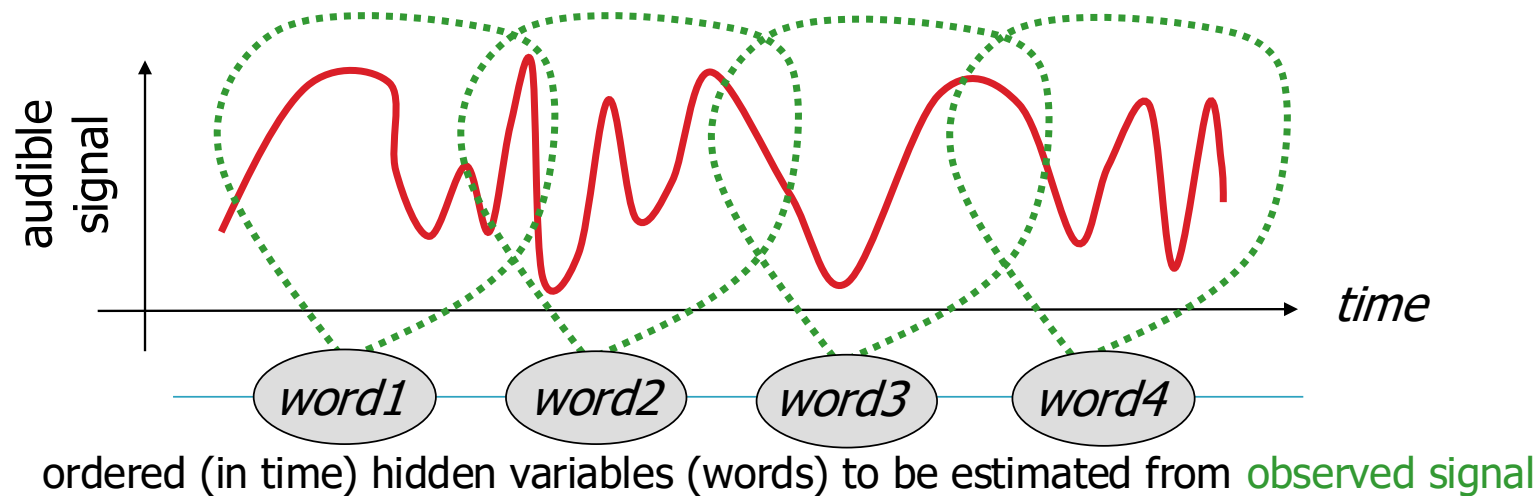
Here we will concentrate on **first-order interactions**

$$E_1(v_1, v_2) + E_2(v_2, v_3) + \dots + E_{n-1}(v_{n-1}, v_n)$$



Dynamic Programming and Hidden Markov Models (HMM)

- DP is widely used in speech recognition

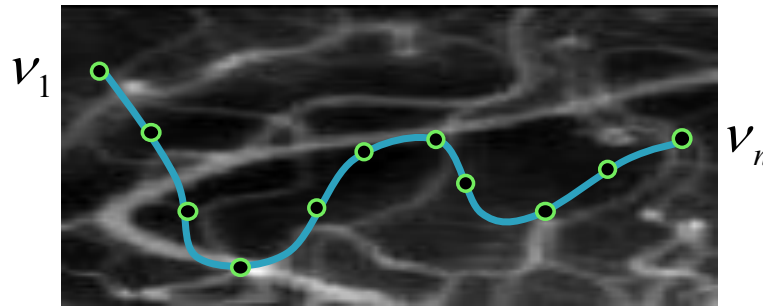


$$E_1(v_0, v_1) + \dots + E_i(v_{i-1}, v_i) + \dots + E_n(v_{n-1}, v_n)$$

$$-\ln \{ \Pr(\text{signal}(t_i) | \text{word}_i) \} - \ln \{ \Pr(\text{word}_i | \text{word}_{i-1}) \}$$

Snakes as Hidden Markov Models (HMM)

- Positions of snake nodes are hidden variables
- Timely order is replaced with spatial order
- Observed audible signal is replaced with image



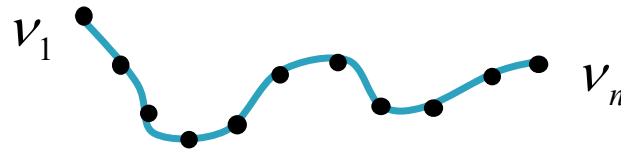
$$E_1(v_1, v_2) + \dots + E_i(v_i, v_{i+1}) + \dots + E_{n-1}(v_{n-1}, v_n)$$

$$- \|\nabla I(v_i)\| + E_{elastic}(v_i, v_{i+1})$$

DP for a Closed Snake?

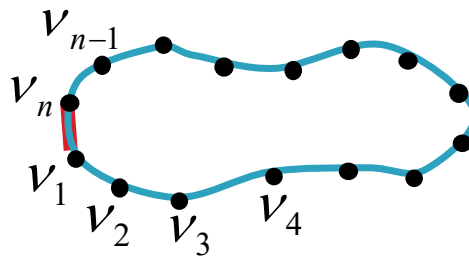
Clearly, DP can be applied to optimize an open ended snake

$$E_1(v_1, v_2) + E_2(v_2, v_3) + \dots + E_{n-1}(v_{n-1}, v_n)$$



Can we use DP for a “looped” energy in case of a closed snake?

$$E_1(v_1, v_2) + E_2(v_2, v_3) + \dots + E_{n-1}(v_{n-1}, v_n) + E_n(v_n, v_1)$$



DP for a Closed Snake?

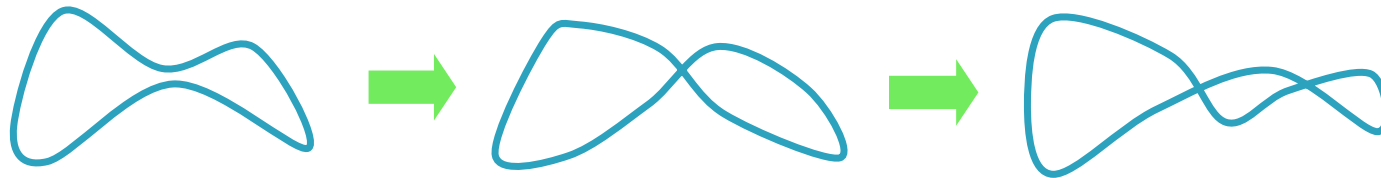
$$E_1(v_1, v_2) + E_2(v_2, v_3) + \dots + E_{n-1}(v_{n-1}, v_n) + E_n(v_n, v_1)$$

Unfortunately, DP can not be directly applied in case of a “loop”.
However, some (approximation) tricks are often used in practice...

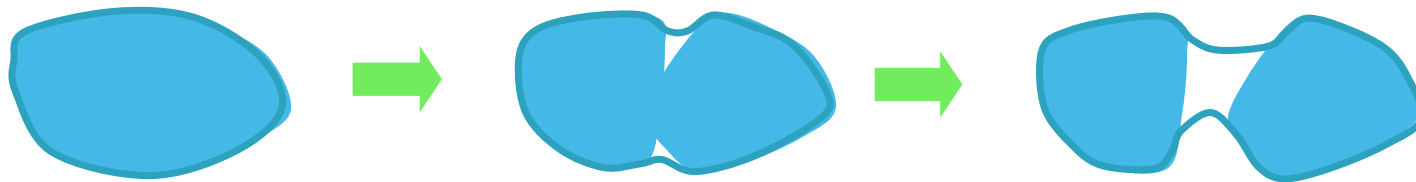
1. Use DP to optimize snake energy with fixed v_1 (according to a given initial snake position).
2. Use DP to optimize snake energy again. This time fix position of an intermediate node $v_{n/2} = \hat{v}_{n/2}$ where $\hat{v}$ is an optimal position obtained in step 1.

Problems with Snakes

- Depends on number and spacing of control points
- Snake may oversmooth the boundary
- Not trivial to prevent curve self intersecting

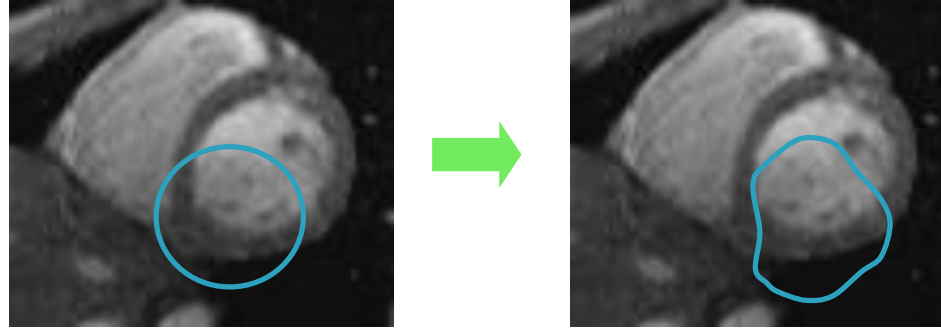


- Can not follow topological changes of objects



Problems with Snakes

- May be sensitive to initialization
 - may get stuck in a local energy minimum near initial contour



- Numerical stability can be an issue for **gradient descent** and **variational methods** (continuous formulation)
 - E.g., requires computing second order derivatives
- The general concept of snakes (deformable models) does generalize to 3D (deformable mesh), but many robust optimization methods suitable for 2D snakes do not apply in 3D
 - E.g.: **dynamic programming** only works for 2D snakes

Problems with Snakes

- External energy: may need to diffuse image gradients, otherwise the snake does not really “see” object boundaries in the image unless it gets very close to it.

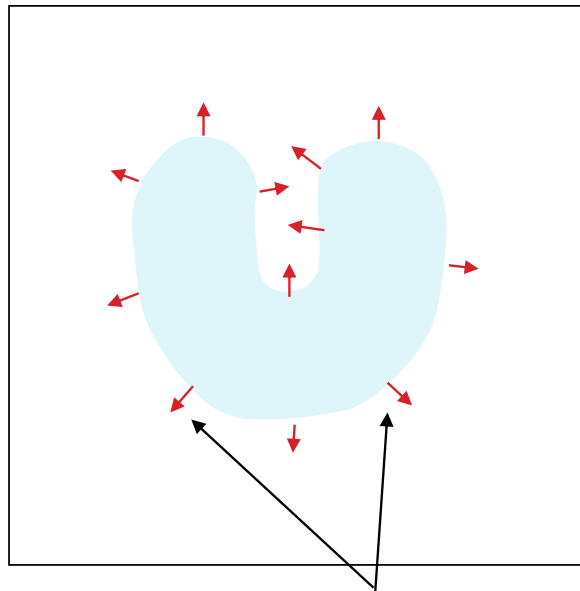
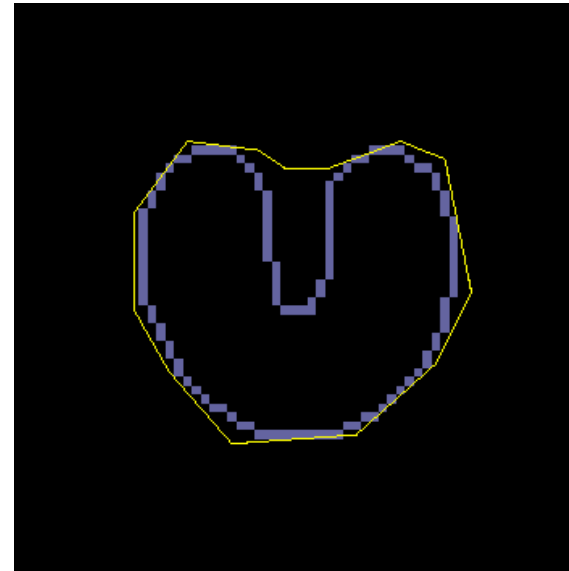


image gradients ∇I
are large only directly on the boundary

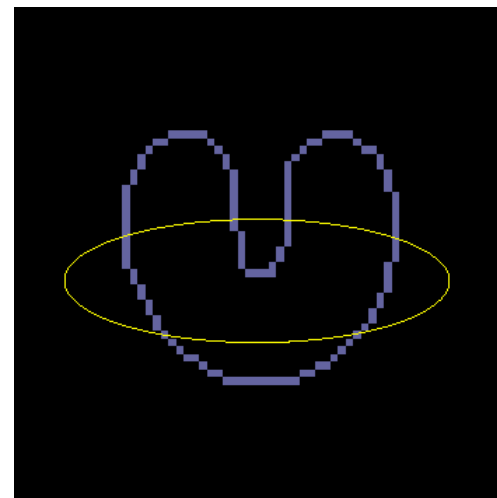
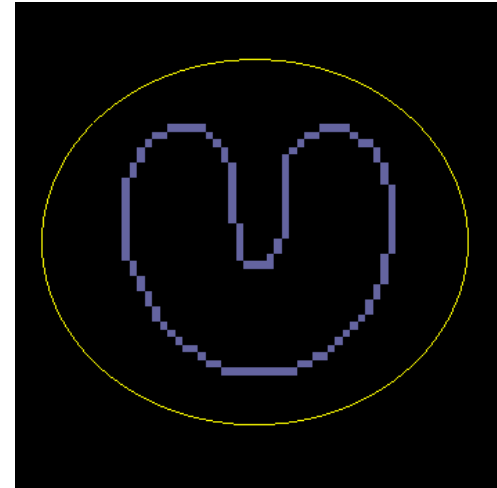


Diffusing Image Gradients

$$\nabla I$$

image gradients diffused via
Gradient Vector Flow (GVF)

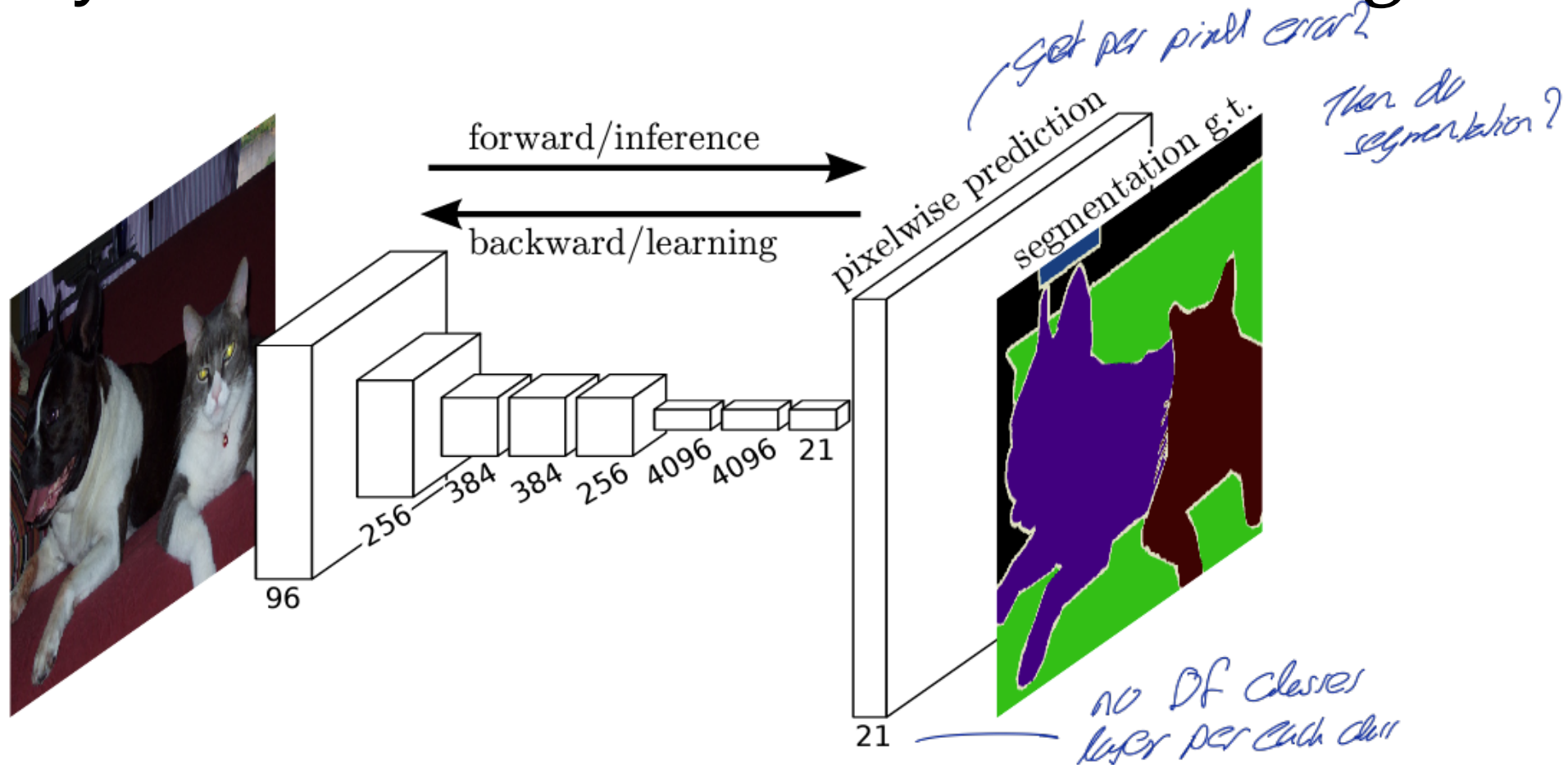
Chenyang Xu and Jerry Prince, 98
<http://iac1.ece.jhu.edu/projects/gvf/>



Other Segmentation Methods

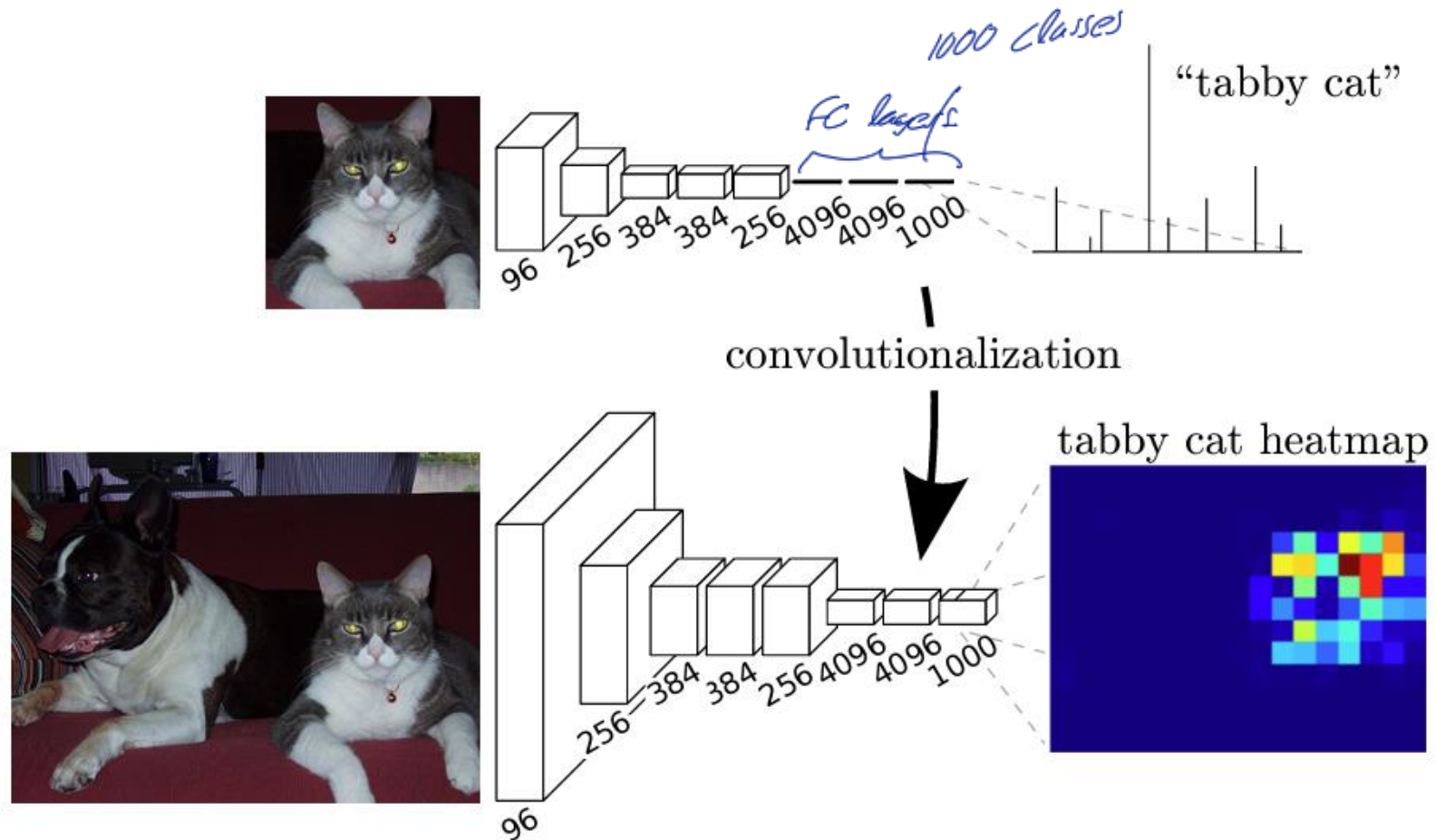
- Watersheds based segmentation (a basic method)
- Level-set based segmentation
- Graph-cut based segmentation
- Semantic segmentation
- Semantic segmentation with deep networks

Fully Convolutional Networks for Sem. Seg.



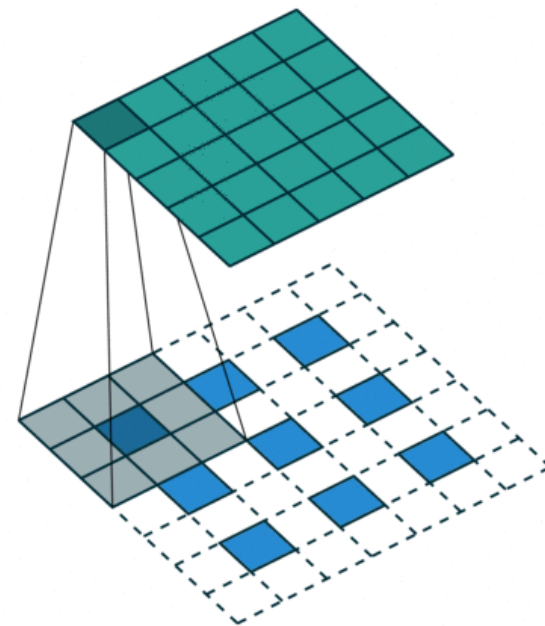
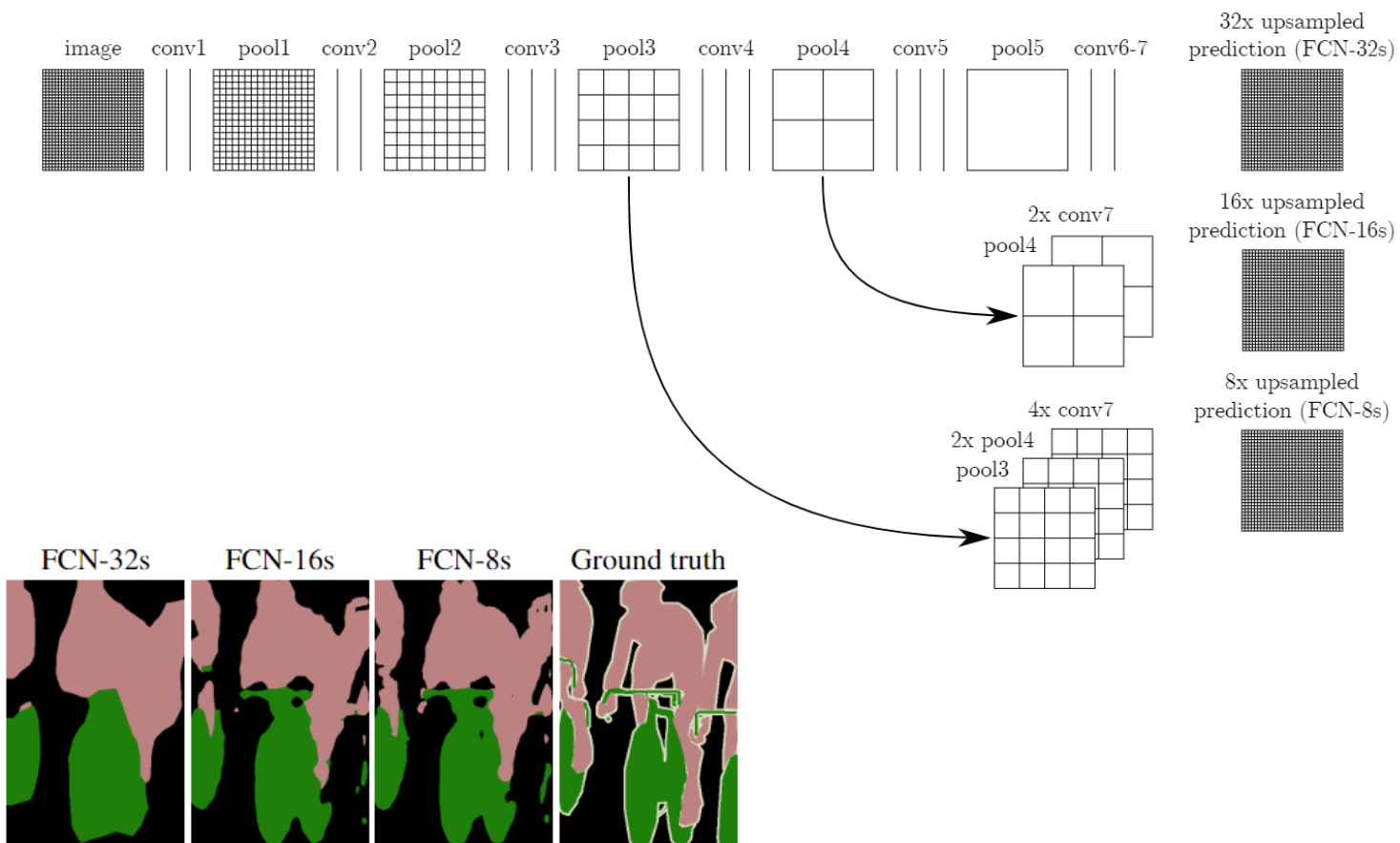
Fully convolutional networks can efficiently learn to make dense predictions for per-pixel tasks like semantic segmentation.

Fully Convolutional Networks for Sem. Seg.



Transforming fully connected layers into convolution layers enables a classification net to output a heat map. Adding layers and aspatial loss (as in Figure 1) produces an efficient machine for end-to-end dense learning

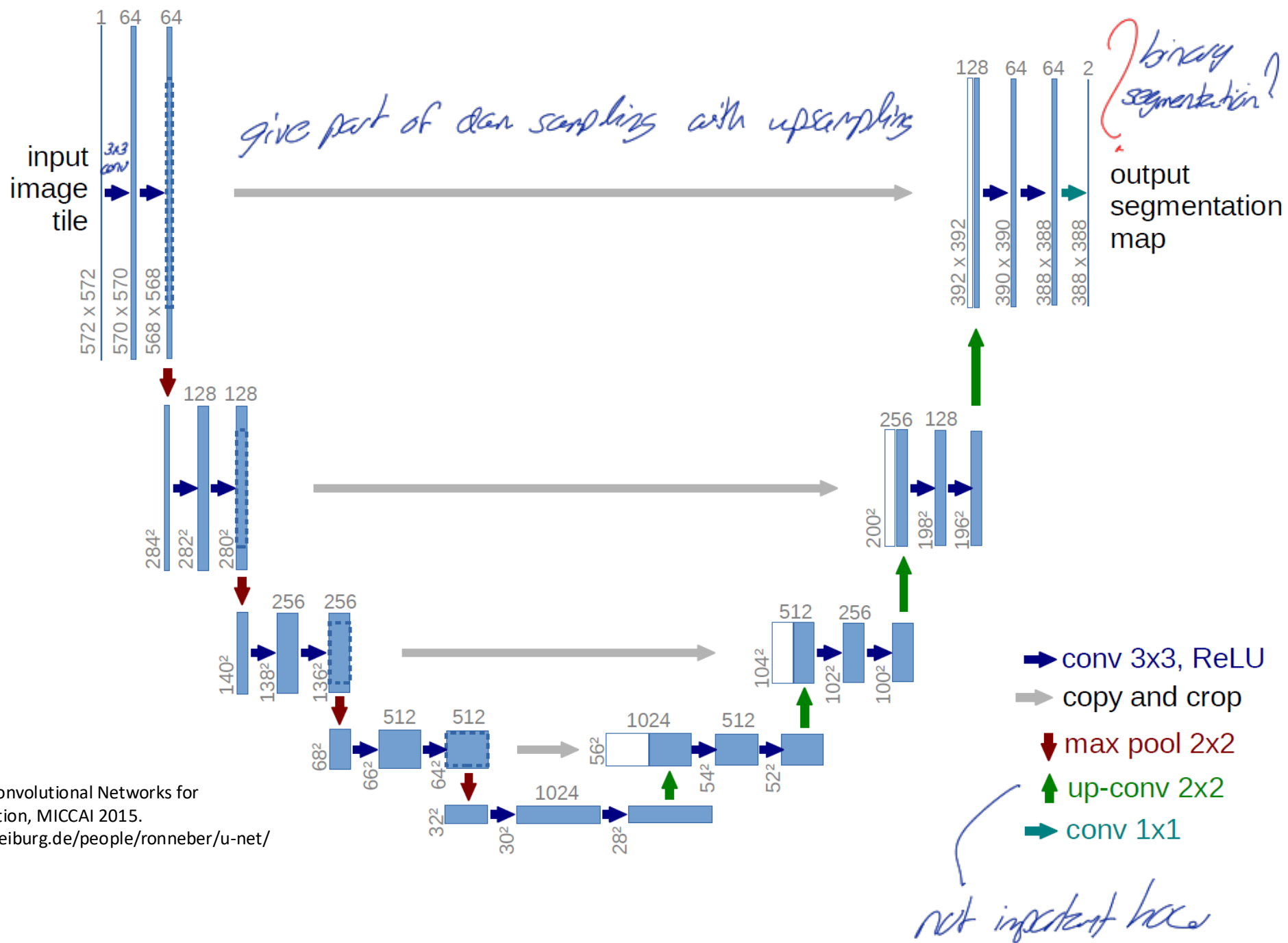
FCN



https://github.com/vdumoulin/conv_arithmetic/blob/master/README.md

Long et al., Fully Convolutional Networks for Semantic Segmentation, CVPR 2015.

U-Net



Ronneberger et al., U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015.
<https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>



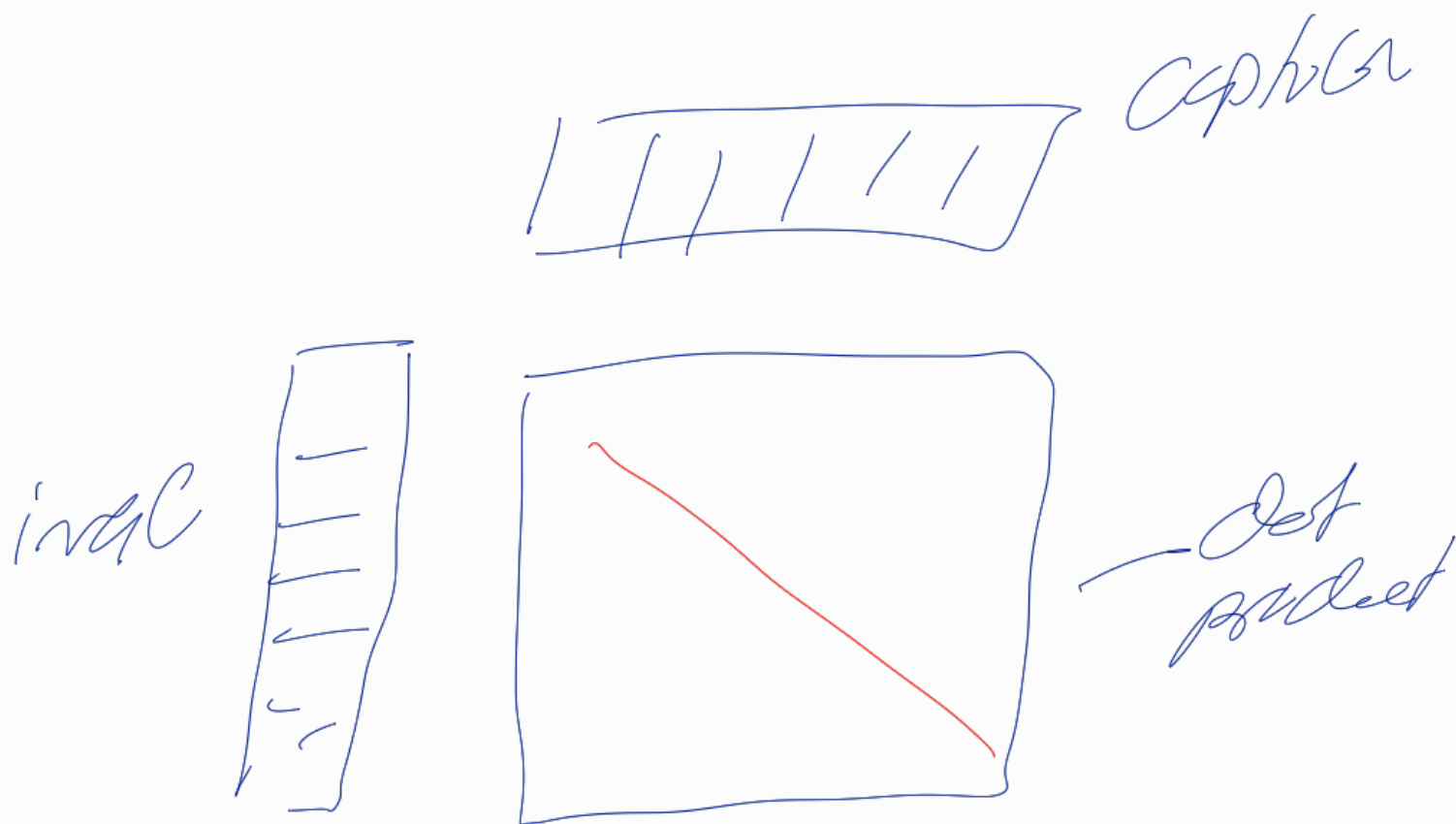
U-net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, and Thomas Brox

Computer Science Department and
BIOSS Centre for Biological Signalling Studies
University of Freiburg, Germany

CLIP by Open AI

we try to match image with caption



if matches perfectly the output is highest and others 0

This is 0 dot. any, when img is given it takes dot product with each and gives the each with highest score

No need to train for new image.
Just provide and get the
output matching word.

